

AI-Powered Multi-Modal Content Generation with Adaptive Voice Cloning and Lip Synchronization

Project Team

Amin Fahim	22I-0618
Sarim Farooq	22i-0458
Musab Waseem	22i-0488

Session 2022-2026

Supervised by

Mr. Muhammad Aamir Gulzar

Co-Supervised by

Dr. Muhammad Asif Naeem



Department of AI & Data Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

December, 2025

Contents

1	Introduction	1
1.1	Existing Solutions	2
1.2	Problem Statement	3
1.3	Scope	4
1.4	Modules	5
1.4.1	Module 1: User and Profile Management	5
1.4.2	Module 2: Speech-to-Text and Text Refinement	6
1.4.3	Module 3: Adaptive Voice Cloning	6
1.4.4	Module 4: Lip Synchronization	6
1.4.5	Module 5: Video Generation and Template Management	6
1.4.6	Module 6: User Interface and User Experience	7
1.4.7	Module 7: Deployment and Evaluation	7
1.5	Work Division	7
2	Project Requirements	9
2.1	Use-case Diagram	9
2.2	Functional Requirements	9
2.2.1	Module 1: User and Profile Management	9
2.2.2	Module 2: Speech-to-Text and Text Refinement	11
2.2.3	Module 3: Adaptive Voice Cloning	11
2.2.4	Module 4: Lip Synchronization	12
2.2.5	Module 5: Video Generation and Template Management	12
2.2.6	Module 6: User Interface	12
2.2.7	Module 7: Deployment and Evaluation	13
2.3	Non-Functional Requirements	13
2.3.1	Reliability	13
2.3.2	Usability	14
2.3.3	Performance	14
2.3.4	Security	15
2.3.5	Scalability	15
2.3.6	Maintainability	15
2.4	Domain Model	16

2.4.1	Core Domain Entities	17
2.4.2	Domain Relationships	18
2.4.3	Domain Constraints and Business Rules	19
3	System Overview	21
3.1	Architectural Design	21
3.2	Design Models	24
3.2.1	Data Flow Diagram - Level 0 (Context Diagram)	24
3.2.2	Data Flow Diagram - Level 1	24
3.2.3	System Sequence Diagrams	25
3.2.3.1	Module 0: High-Level Workflow Overview	25
3.2.3.2	Module 1: User & Profile Management	27
3.2.3.3	Module 2: Speech-to-Text Transcription & Text Refinement	28
3.2.3.4	Module 3: Adaptive Voice Cloning	29
3.2.3.5	Module 4: Voice Synthesis & Text-to-Speech	30
3.2.3.6	Module 5: Lip Synchronization & Video Generation	31
3.2.3.7	Module 6: Preview, Export & Content Delivery	32
3.2.4	Database Schema	32
3.2.5	File Storage Organization	35
3.2.6	Data Processing Pipeline	35
3.3	Conclusion	36
3.4	Future Work	36
4	Implementation and Testing	39
4.1	General Description	39
4.1.1	System Context	39
4.1.2	Technology Stack	39
4.1.3	Current Implementation Status	40
4.2	Algorithm Design	40
4.2.1	User Authentication and Synchronization	40
4.2.2	Iterative Script Refinement with Feedback	43
4.2.3	Script Refinement (Simple and Custom)	44
4.2.4	Voice Cloning with XTTS v2	44
4.2.5	Custom XTTS v2 Training for Urdu Language	46
4.2.5.1	Dataset Collection and Preprocessing	46
4.2.5.2	Vocabulary Extension with BPE	48
4.2.5.3	XTTS v2 Fine-tuning Pipeline	48
4.2.5.4	Training Environment Challenges	50
4.3	External APIs/SDKs	50
4.4	Testing Details	51

4.4.1	Unit Testing	51
4.4.1.1	Authentication Middleware Test	51
4.4.1.2	Script Generation Metadata Calculation Test	53
4.4.2	Integration Testing	55
4.4.2.1	User Synchronization Flow Test	55
4.4.2.2	Script Generation End-to-End Test	56
4.4.3	Manual Testing	58
4.4.3.1	Dashboard Navigation Test	58
4.4.3.2	Script Generation User Flow Test	59
4.4.4	Security Testing	60
4.4.4.1	Authentication Security Test	60
4.4.5	XTTS v2 Voice Cloning Testing	61
4.4.5.1	English Voice Cloning Test	61
4.4.5.2	Urdu Fine-tuned Model Test	63
4.4.6	Test Coverage Summary	66
5	Conclusions and Future Work	69
5.1	Conclusion	69
5.2	Future Work	70
5.2.1	Improving Urdu Voice Cloning	70
5.2.2	Lip Synchronization and Video Generation	70
	References	72

List of Figures

2.1	Use Case Diagram showing primary user interactions	10
3.1	High-level Microservices Architecture showing service interactions and communication	21
3.2	Data Flow Diagram Level 0 - Context Diagram	24
3.3	Data Flow Diagram Level 1 - Major Processes	25
3.4	Module 0 - High-level end-to-end workflow showing interaction between all system modules	26
3.5	Module 1 - User authentication, session management, and profile initialization	27
3.6	Module 2 - Audio transcription and language model-based text enhancement	28
3.7	Module 3 - Voice model training and adaptive voice cloning for personalized synthesis	29
3.8	Module 4 - Text-to-speech conversion using cloned voice models	30
3.9	Module 5 - Lip-sync alignment and final video generation	31
3.10	Module 6 - Video preview, download management, and CDN distribution	32

List of Tables

1.1	Comparison of Existing Solutions	3
1.2	Work Division and Task Allocation	8

Chapter 1

Introduction

The rapid evolution of artificial intelligence has fundamentally transformed digital media production, yet the workflow for personalized content creation remains fragmented and resource-intensive. This project, titled *AI-Powered Multi-Modal Content Generation with Adaptive Voice Cloning and Lip Synchronization*, aims to bridge these gaps by developing a unified system capable of generating realistic, single-speaker videos from simple audio or text inputs.

Background

Content creation in the modern digital landscape is often time-consuming, skill-intensive, and reliant on expensive, specialized software. The increasing demand for personalized digital content across platforms such as online education, social media, news broadcasting, and marketing has amplified the need for efficient and intelligent automation in media production pipelines. As creators aim to scale their output while maintaining quality and authenticity, traditional manual workflows are becoming increasingly unsustainable.

Recent advances in Deep Learning and generative artificial intelligence have significantly improved individual components of the content creation process, including Automatic Speech Recognition (ASR), neural text generation, speech synthesis, and computer vision-based facial animation. Models such as transformer-based ASR systems and neural vocoders have enabled highly accurate transcription and natural-sounding speech, while deep generative models have dramatically improved the realism of lip synchronization and facial expressions. Despite these advancements, these technologies largely exist as isolated solutions rather than as components of a unified system.

While progress has been made in multilingual and multi-modal AI systems, there is currently no single integrated solution that addresses the complete workflow for bilingual content creation, specifically for **Urdu and English** speakers. Urdu, as a low-resource

language in the AI ecosystem, presents additional challenges due to limited high-quality annotated datasets, script complexity, and phonetic variability. Consequently, content creators are often forced to rely on Romanized text, third-party translation tools, or manual voice recording, which compromises authenticity and linguistic accuracy.

The motivation for this project stems from several critical limitations in the current ecosystem:

1. **Fragmented Workflows:** Creators are forced to switch between multiple platforms for transcription, text refinement, voice synthesis, and video generation, leading to inefficiencies, increased production time, and quality inconsistencies [12].
2. **Lack of Low-Resource Language Support:** Most existing high-end content generation tools lack native support for Urdu, requiring users to rely on poor Roman transliterations or manual translation processes that reduce expressiveness and cultural fidelity.
3. **Static Voice Models:** Current voice cloning solutions often lack adaptivity, failing to improve the voice model progressively with user interaction and feedback, which limits personalization and long-term usability.

To address these challenges, this project proposes a unified framework that integrates Speech-to-Text (STT), Large Language Model (LLM)-based text refinement, adaptive voice cloning, and visual lip synchronization into a cohesive and automated pipeline. By combining these components within a single system, the proposed approach aims to reduce technical barriers, improve workflow efficiency, and enable high-quality bilingual content generation with minimal user intervention.

1.1 Existing Solutions

To contextualize the proposed system, it is essential to analyze existing methods in the domain of multi-modal content generation. Currently, the landscape is dominated by isolated tools that excel in specific tasks—such as transcription, voice synthesis, or visual synchronization—but fail to offer an end-to-end user experience.

For instance, robust speech recognition models like Whisper [10] handle transcription with high accuracy but offer no generation capabilities. Similarly, advanced voice cloning frameworks like OpenVoice [9] focus on zero-shot synthesis but do not integrate directly with visual lip-synchronization pipelines like Wav2Lip [8] in a user-friendly manner. This disjointed nature requires significant technical expertise to bridge the components manually. The following table compares these prominent existing technologies with the objectives of the proposed system.

Table 1.1: Comparison of Existing Solutions

System Name	System Overview	System Limitations
OpenVoice [9]	A versatile instant voice cloning approach that can replicate a speaker's voice color and tone from a short audio reference (zero-shot learning).	It functions primarily as a standalone audio synthesis tool. It lacks integration with visual modules (lip-sync) and does not natively support the progressive "adaptive" learning loop for user-specific voice refinement proposed in this project.
Wav2Lip [8]	A deep learning-based lip-synchronization model that accurately syncs a target video's lip movements to any input audio stream.	It is strictly a visual processing component. It requires pre-generated audio and video inputs and offers no text refinement, transcription, or voice cloning capabilities, forcing users to generate audio elsewhere.
Whisper [10]	A state-of-the-art automatic speech recognition (ASR) system that provides robust multilingual transcription and translation.	While accurate for STT, it is uni-modal (text-only output). It cannot generate audio or video content and lacks the ability to refine scripts for coherence or format them for teleprompter/video usage.

1.2 Problem Statement

The rapid growth of digital media platforms has significantly increased the demand for personalized and multilingual video content across education, marketing, news delivery, and social media. However, the process of producing high-quality content remains time-consuming, technically demanding, and heavily dependent on expensive and specialized software tools [6]. While recent advances in artificial intelligence have introduced powerful solutions for individual tasks such as speech recognition, text generation, voice synthesis, and video editing, these tools largely operate as isolated components rather than as a unified system [14].

This fragmentation creates major inefficiencies in the content creation workflow, requiring

users to switch between multiple platforms and manually integrate outputs at each stage. Such workflows not only increase production time but also introduce quality inconsistencies and usability barriers, particularly for users without advanced technical expertise [5]. The problem is further compounded in the context of bilingual content creation, where most existing systems lack native support for low-resource languages such as Urdu, forcing creators to rely on inaccurate transliterations or manual translations [7]. As a result, linguistic authenticity, pronunciation accuracy, and expressive quality are often compromised.

Another critical limitation of current solutions is the static nature of voice cloning models, which typically do not improve over time or adapt to user-specific speech patterns and feedback [3]. This restricts long-term personalization and reduces the practical value of such systems for recurring content generation. Consequently, there exists a significant gap in the availability of an integrated, adaptive, and language-inclusive solution that can automate the complete pipeline of single-speaker content generation.

This project aims to address these challenges by developing an AI-powered, end-to-end system that unifies speech-to-text transcription, intelligent script refinement, adaptive voice cloning, and visual lip synchronization within a single coherent framework [13]. By streamlining the workflow and removing technical barriers, the proposed system enables creators, educators, and businesses to generate professional-quality bilingual video content efficiently, consistently, and with minimal manual intervention. The successful implementation of this system contributes to the field of multi-modal content generation by improving accessibility for low-resource languages and advancing adaptive personalization in AI-driven media production.

1.3 Scope

The scope of this project defines the functional and technical boundaries of the proposed AI-powered content generation system. The primary objective of the system is to support automated, single-speaker video generation with native bilingual support for Urdu and English languages [11]. The system is designed to prioritize usability and efficiency while maintaining high-quality audio-visual output, making it accessible to users with limited technical expertise [1].

The project focuses on offline or semi-automated content generation rather than real-time live synthesis. Users will be able to upload audio recordings or short video templates of up to five minutes in duration, which will serve as input for speech processing, voice modeling, and lip synchronization [8]. The generated output will be exported as standard video files suitable for direct publishing across common digital platforms.

- **What the project will cover:** The system will include accurate speech-to-text transcription for both Urdu and English audio inputs using state-of-the-art ASR models [10]. It will incorporate Large Language Model-based text refinement to improve script clarity, coherence, and suitability for spoken delivery [9]. The project will implement adaptive voice cloning that learns from user-provided voice samples and incrementally improves synthesis quality over continued use [9]. Additionally, the system will support template-based lip synchronization by aligning generated speech with facial movements in provided video inputs. User profile management will be included to persist voice samples, configuration parameters, and historical generation data.
- **What the project will not cover:** The scope excludes multi-speaker dialogue generation and interactions involving multiple avatars or speakers within a single video [4]. Advanced avatar animation techniques such as full-body motion capture, emotional gesture modeling, or real-time facial expression synthesis are outside the scope of this project. The system will not address large-scale distributed model training or cloud-based deployment optimization, focusing instead on efficient inference for individual users. Post-deployment commercial scalability, content moderation, and security hardening are also not considered in the current scope.

1.4 Modules

The proposed system is decomposed into seven interconnected modules, each with clearly defined responsibilities to ensure modularity and maintainability [2]. These modules work sequentially to transform raw user input into polished, lip-synced video content, handling everything from authentication to final rendering.

1.4.1 Module 1: User and Profile Management

This module provides a minimal but secure account management system to persist user settings, voice samples, and preferred video templates [2]. It acts as the gatekeeper for the system, ensuring data privacy and organizing user-specific datasets for model training [2].

1. Secure user registration and authentication handling.
2. Management of profile-linked datasets including audio samples and text transcripts.
3. Implementation of consent tracking mechanisms for voice data usage and storage.

1.4.2 Module 2: Speech-to-Text and Text Refinement

This module handles bilingual transcription and intelligent script enhancement by integrating state-of-the-art speech-to-text models [10]. It focuses on converting mixed-language audio into structured text and refining it for clarity without relying on Roman transliteration [11].

1. Accurate bilingual (Urdu and English) speech-to-text transcription.
2. Language model-based text cleanup to improve formatting and coherence.
3. Generation of structured, punctuated scripts from raw audio inputs.

1.4.3 Module 3: Adaptive Voice Cloning

This module implements progressive voice synthesis technology that adapts and improves based on user interaction [9]. It utilizes few-shot and zero-shot learning approaches to create high-fidelity voice clones that evolve as users provide more data [4].

1. Implementation of few-shot and zero-shot voice cloning algorithms.
2. Persistent per-user voice models that improve with additional training data.
3. Voice quality evaluation using Mean Opinion Score (MOS) metrics.

1.4.4 Module 4: Lip Synchronization

This module is responsible for synchronizing the synthesized audio with facial movements in video templates to ensure visual realism [8]. It integrates models like Wav2Lip and applies post-processing techniques to maintain high visual quality [15].

1. Synchronization of generated speech with facial landmarks in video templates.
2. Implementation of alignment quality checks and adjustment heuristics.
3. Optional video enhancement and upscaling using tools like RealESRGAN.

1.4.5 Module 5: Video Generation and Template Management

This module manages the final video assembly, handling the composition of audio, video, and background elements [2]. It provides a library of customizable templates and renders the final output in standard formats suitable for publishing [2].

1. Management of a library of video templates with customizable backgrounds.
2. Robust audio-video composition and rendering pipeline.
3. Export functionality for standard video formats (MP4, MOV).

1.4.6 Module 6: User Interface and User Experience

This module provides the front-end workflows for all user interactions, ensuring the system is accessible to non-technical creators [2]. It streamlines the upload and review process while providing comprehensive feedback and error handling [2].

1. Intuitive, drag-and-drop interfaces for media upload and content review.
2. Real-time progress indicators and comprehensive error messaging.
3. Support for iterative UX improvements based on usability testing.

1.4.7 Module 7: Deployment and Evaluation

This module focuses on the operationalization of the system, establishing pipelines for continuous integration and deployment [2]. It also manages system monitoring, latency checks, and automated quality assurance for audio outputs [2].

1. Establishment of CI/CD pipelines for automated deployment.
2. Implementation of latency monitoring and audio quality checks.
3. Maintenance of comprehensive test plans and documentation.

1.5 Work Division

The project workload has been distributed among the three team members to ensure efficient execution of all modules. The tasks are divided based on individual distinct roles in research, model training, and full-stack system development.

Table 1.2: Work Division and Task Allocation

Member Name	Roll Number	Assigned Responsibilities
Sarim Farooq	22i-0458	• Research for voice cloning architectures. • Testing different approaches for voice cloning. • English TTS testing and validation. • Testing different approaches for model training optimization.
Amin Fahim	22i-0618	• Data collection for Urdu TTS training. • Preprocessing raw audio and text data for training. • Creating the automated pipeline for model training. • Training the specific TTS models for Urdu language support.
Musab Waseem	22i-0488	• User profile management and authentication implementation. • Development of the Script Generation module in Next.js and Node.js. • API creation and backend integration for all services.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the proposed AI-powered content generation system.

2.1 Use-case Diagram

The system follows a use case-driven approach given its interactive, end-user-focused nature. The use case diagram illustrates the primary interactions between different user classes and the system functionalities.

The primary actors include Content Creators, Educators, Influencers, and Business Users who interact with the system through registration, media upload, voice cloning, text refinement, lip-sync video generation, and content export functionalities. The system administrator manages user accounts, templates, and system configuration.

2.2 Functional Requirements

This section describes the functional requirements organized by system modules, expressed in natural language for clarity and traceability.

2.2.1 Module 1: User and Profile Management

Following are the functional requirements for user and profile management:

1. The system shall allow new users to register with email and password authentication.

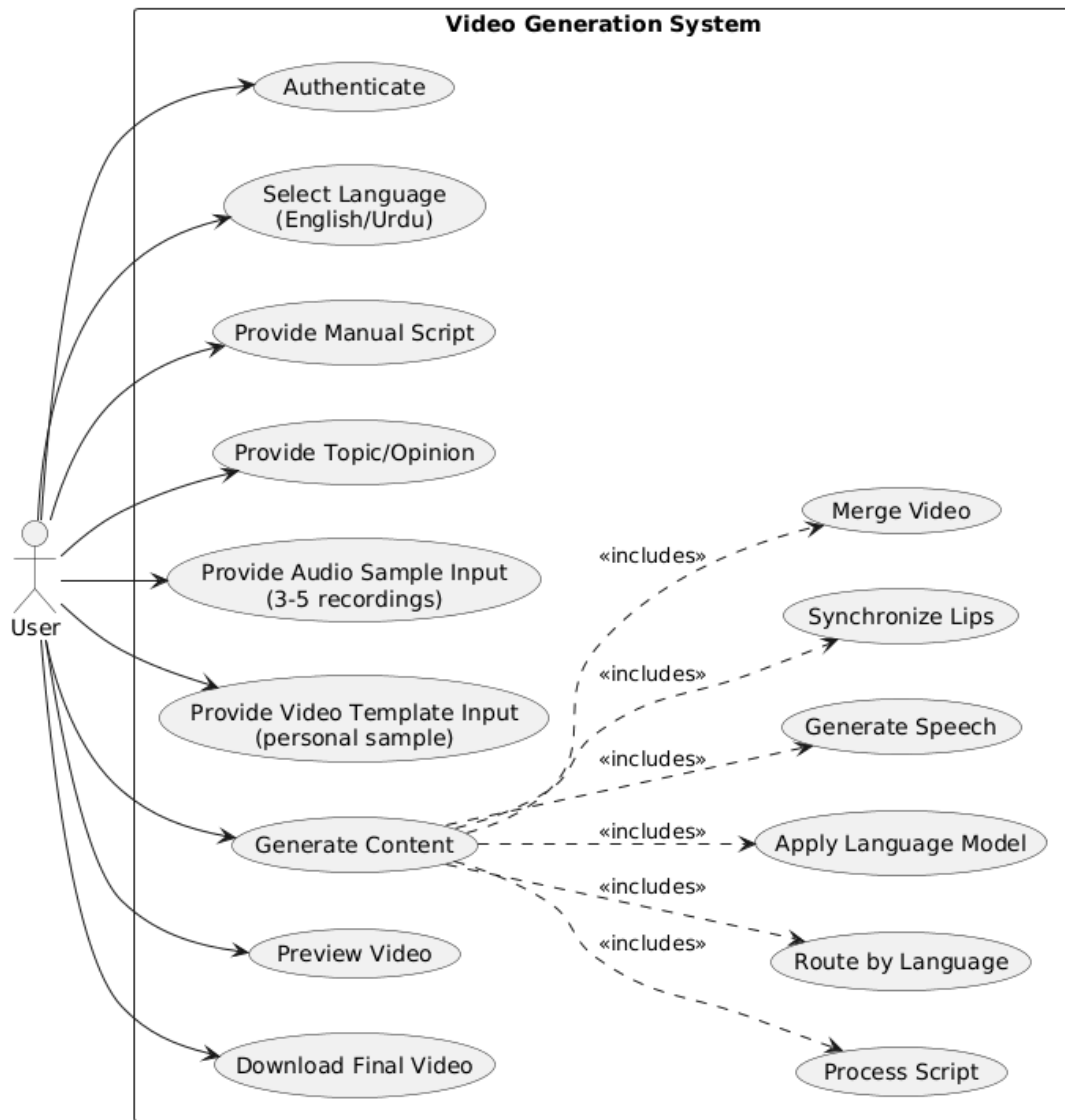


Figure 2.1: Use Case Diagram showing primary user interactions

2. The system shall securely store user credentials using industry-standard encryption.
3. The system shall maintain user profiles containing voice samples, preferred templates, and historical data.
4. The system shall implement session management with automatic logout after inactivity.
5. The system shall track and store user consent for voice data usage and storage.
6. The system shall allow users to view, update, and delete their profile information.

2.2.2 Module 2: Speech-to-Text and Text Refinement

Following are the functional requirements for STT and text processing:

1. The system shall transcribe Urdu and English audio with minimum 85% accuracy.
2. The system shall support mixed-language audio inputs without requiring transliteration.
3. The system shall automatically detect the language of input audio.
4. The system shall refine transcribed text using language models to improve structure and clarity.
5. The system shall generate formatted scripts with proper punctuation and paragraph breaks.
6. The system shall allow users to review and edit refined transcripts before processing.
7. The system shall store both original and refined transcripts for user reference.

2.2.3 Module 3: Adaptive Voice Cloning

Following are the functional requirements for voice cloning functionality:

1. The system shall generate synthetic speech that closely matches the user's voice characteristics.
2. The system shall support few-shot voice cloning with minimal input samples (3-5 recordings).
3. The system shall implement zero-shot voice cloning for rapid prototyping.
4. The system shall maintain persistent user voice models that improve with additional samples.
5. The system shall allow users to approve or reject generated voice outputs.
6. The system shall calculate and display voice quality metrics for user feedback.
7. The system shall support both Urdu and English speech synthesis in cloned voices.

2.2.4 Module 4: Lip Synchronization

Following are the functional requirements for lip-sync functionality:

1. The system shall synchronize generated audio with facial movements in user-provided videos.
2. The system shall support multiple video template formats (MP4, AVI, MOV).
3. The system shall detect and track facial landmarks for accurate lip synchronization.
4. The system shall implement quality checks to ensure natural-looking lip movements.
5. The system shall provide optional video enhancement and upscaling.
6. The system shall handle videos with varying resolutions and aspect ratios.

2.2.5 Module 5: Video Generation and Template Management

Following are the functional requirements for video generation:

1. The system shall maintain a library of customizable video templates.
2. The system shall allow users to upload custom video templates.
3. The system shall compose audio and video streams into final output files.
4. The system shall export videos in standard formats (MP4, MOV) suitable for publishing.
5. The system shall support background replacement and overlay customization.
6. The system shall render videos with configurable quality settings (720p, 1080p).

2.2.6 Module 6: User Interface

Following are the functional requirements for the user interface:

1. The system shall provide a web-based interface accessible via modern browsers.
2. The system shall implement drag-and-drop functionality for media uploads.
3. The system shall display real-time progress indicators during processing.

4. The system shall provide preview functionality for generated content before final export.
5. The system shall implement comprehensive error messages with suggested solutions.
6. The system shall support responsive design for desktop and tablet devices.

2.2.7 Module 7: Deployment and Evaluation

Following are the functional requirements for deployment and testing:

1. The system shall implement automated testing for all core functionalities.
2. The system shall monitor and log system performance metrics.
3. The system shall provide API documentation for all backend services.
4. The system shall implement continuous integration and deployment pipelines.
5. The system shall generate quality assessment reports for audio and video outputs.

2.3 Non-Functional Requirements

This section specifies quality attributes and constraints that the system must satisfy.

2.3.1 Reliability

REL-1: The system shall maintain 95% uptime during operational hours, with mean time between failures (MTBF) exceeding 720 hours under normal load conditions.

REL-2: The system shall implement automatic recovery mechanisms for transient failures in transcription and voice cloning modules.

REL-3: The system shall provide graceful degradation when third-party services are unavailable, notifying users of reduced functionality.

REL-4: The system shall persist user data and processing state to prevent data loss during unexpected shutdowns.

2.3.2 Usability

USE-1: The system shall allow users to initiate content generation with no more than three distinct interactions from the main dashboard.

USE-2: The system shall provide context-sensitive help and tooltips for all major features.

USE-3: New users shall be able to generate their first video output within 15 minutes of account creation, including voice sample collection.

USE-4: The system shall support undo/redo functionality for text editing operations.

USE-5: Error messages shall be displayed in both English and Urdu based on user preference.

2.3.3 Performance

PER-1: The system shall transcribe up to 5 minutes of audio within 30 seconds on standard hardware.

PER-2: Text refinement using language models shall complete within 10 seconds for typical input lengths (500-1000 words).

PER-3: Voice cloning inference shall generate speech at a rate of at least 1:1 (real-time) relative to audio duration.

PER-4: Lip synchronization processing shall complete within 2 minutes for 1 minute of source video at 1080p resolution.

PER-5: The web interface shall load initial page content within 2 seconds over a 10 Mbps connection.

PER-6: The system shall support concurrent processing of up to 10 user requests without significant performance degradation.

2.3.4 Security

SEC-1: The system shall encrypt all user passwords using bcrypt or equivalent hashing with minimum 12-round salting.

SEC-2: The system shall implement HTTPS for all client-server communications.

SEC-3: The system shall sanitize all user inputs to prevent SQL injection and cross-site scripting attacks.

SEC-4: User voice samples and generated content shall be stored with user-specific access controls.

SEC-5: The system shall implement API rate limiting to prevent abuse (100 requests per hour per user).

SEC-6: The system shall log all authentication attempts and flag suspicious activities.

2.3.5 Scalability

SCA-1: The system architecture shall support horizontal scaling to accommodate increasing user loads.

SCA-2: Database design shall accommodate at least 10,000 registered users with minimal performance impact.

SCA-3: Media storage shall be designed to handle growth to 1TB within the first year of operation.

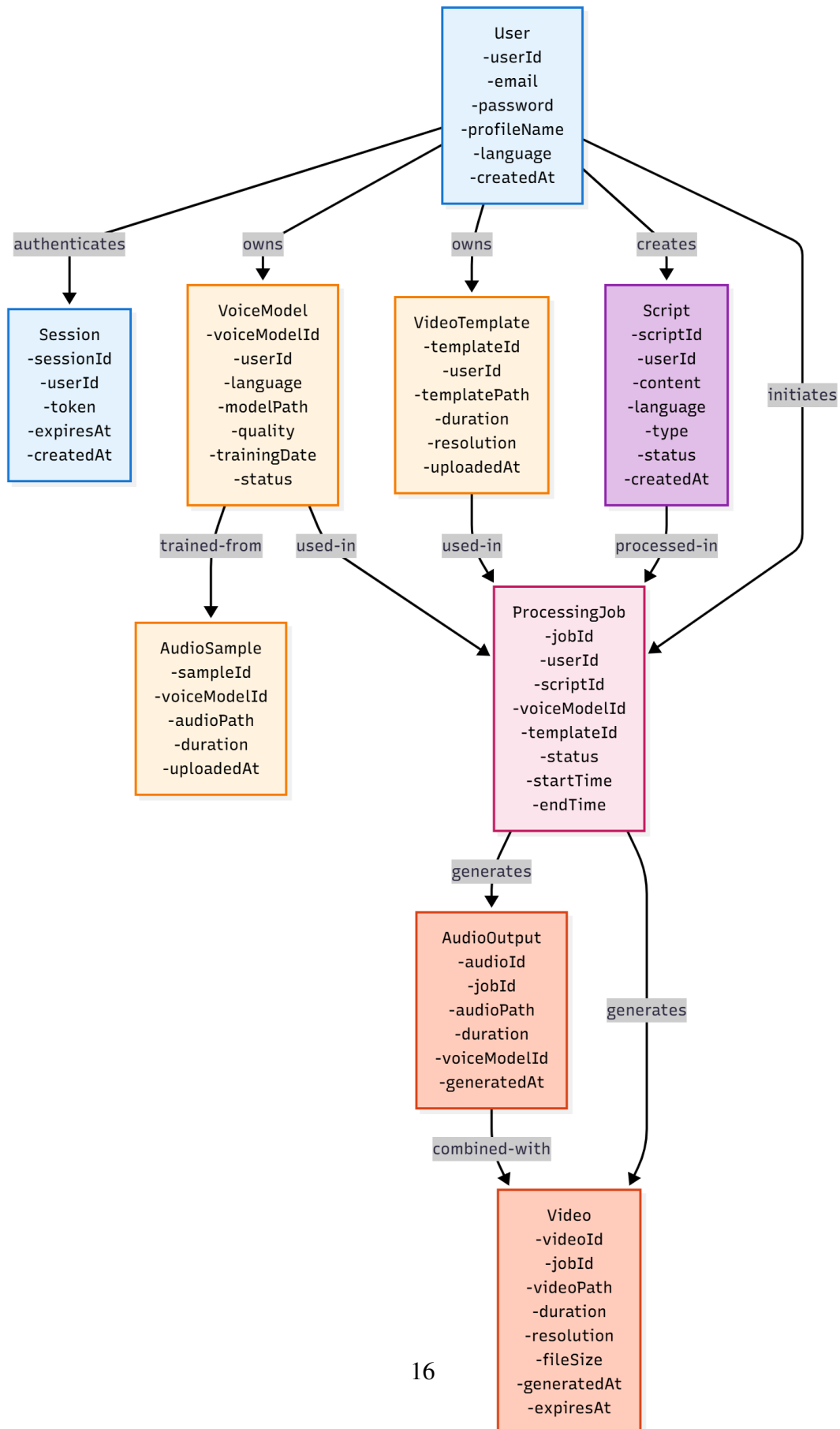
2.3.6 Maintainability

MAI-1: The codebase shall maintain minimum 70% unit test coverage for core modules.

MAI-2: All modules shall follow consistent coding standards and documentation practices.

MAI-3: The system shall implement structured logging for debugging and monitoring.

2.4 Domain Model



The domain model represents the core business entities and their relationships within the bilingual AI video generation system. It defines the fundamental concepts, their properties, and how they interact to support the video creation workflow.

2.4.1 Core Domain Entities

User: The User entity represents an authenticated user of the system. It maintains essential user information including `userId` (unique identifier), email address, password hash for authentication, profile name for display purposes, language preference (English or Urdu), and account creation timestamp. Each user is the central entity around which all other domain objects are organized.

Session: The Session entity manages user authentication state and active sessions. It contains a `sessionId` for unique identification, `userId` to link the session to a specific user, an authentication token for request validation, an expiration timestamp to enforce session timeouts, and creation timestamp for audit purposes. Sessions ensure secure access control and maintain user login state.

VoiceModel: The VoiceModel entity represents a personalized voice clone for a user in a specific language. Each voice model has a unique `voiceModelId`, `userId` to associate it with the owner, language identifier (English or Urdu), path to the trained model file, quality metric for evaluation, training date, and status indicator showing whether the model is ready for use or still processing. Users maintain separate voice models for each language to support bilingual content creation.

AudioSample: The AudioSample entity represents individual audio recordings provided by users during voice cloning. Each sample has a `sampleId`, reference to its parent `voiceModelId`, path to the audio file, duration in seconds, and upload timestamp. Typically, users provide 3-5 audio samples that collectively train the voice cloning model.

VideoTemplate: The VideoTemplate entity stores the personal video template uploaded by each user for use in lip synchronization. It contains `templateId` for unique identification, `userId` to link it to the owner, path to the template video file, duration in seconds, video resolution specifications, and upload timestamp. The template serves as the visual foundation for all generated videos.

Script: The Script entity represents either user-provided or AI-generated content that will be converted into spoken audio. Each script has a `scriptId`, `userId` for ownership, the actual script content as text, language identifier (English or Urdu), type indicator (generated from topic or manually provided), status tracking (draft, processing, completed), and creation timestamp. Scripts are the primary input for the audio generation process.

ProcessingJob: The ProcessingJob entity orchestrates the entire video generation workflow. It serves as the central control entity that links together a script, voice model, and

video template. **ProcessingJob** contains `jobId` for unique identification, `userId` for the user initiating the job, `scriptId` referencing the script to be processed, `voiceModelId` referencing the user's voice clone, `templateId` referencing the video template, status indicator for job state (pending, processing, completed, failed), start time and end time for duration tracking, and resource usage metrics. A processing job represents one complete video generation request.

AudioOutput: The **AudioOutput** entity represents the synthetic audio generated from a script using a specific voice model. It contains `audioId` for unique identification, `jobId` linking it to the processing job, path to the generated audio file, duration in seconds, `voiceModelId` identifying which voice model was used, and generation timestamp. The audio output is a key intermediate artifact in the video generation pipeline.

Video: The **Video** entity represents the final generated video combining synchronized audio and visual elements. It contains `videoId` for unique identification, `jobId` linking it to the processing job, path to the final video file, duration in seconds, resolution specifications, file size in bytes, generation timestamp, and expiration timestamp for temporary storage management. The video is the final output delivered to the user.

2.4.2 Domain Relationships

The domain entities are connected through the following key relationships:

1. **User Authentication:** User authenticates to create a **Session**, establishing a secure connection for subsequent operations.
2. **User Ownership:** User owns multiple **VoiceModels** (one per language), multiple **VideoTemplates**, and multiple **Scripts**. These are user-specific assets created during account setup and content creation.
3. **Voice Model Training:** **VoiceModel** is trained from multiple **AudioSamples**. Each voice model maintains a reference to all audio samples used in its training.
4. **Job Initialization:** User initiates **ProcessingJob** requests to generate videos. Each job is associated with exactly one user.
5. **Script Processing:** **ProcessingJob** processes a **Script** by using the script's content as input to the text-to-speech pipeline.
6. **Voice Model Usage:** **ProcessingJob** uses a **VoiceModel** to generate synthetic audio with the user's personalized voice.
7. **Template Usage:** **ProcessingJob** uses a **VideoTemplate** to provide the visual foundation for lip synchronization.

8. **Audio Generation:** ProcessingJob generates AudioOutput containing the synthetic speech created from the script using the selected voice model.
9. **Video Generation:** ProcessingJob generates Video by combining AudioOutput with the VideoTemplate through lip synchronization and video processing.
10. **Audio-Video Integration:** AudioOutput is combined with Video to produce the final synchronized output where lip movements match the generated speech.

2.4.3 Domain Constraints and Business Rules

- Each User must have at least one VoiceModel to generate videos. Voice models are language-specific, so users creating bilingual content require separate models for English and Urdu.
- Each User must provide at least one VideoTemplate to define the visual appearance for video generation.
- Each Script must be associated with exactly one language (English or Urdu) and must match the language of the selected VoiceModel in the ProcessingJob.
- ProcessingJob requires valid references to a Script, VoiceModel, and VideoTemplate before processing can commence.
- AudioOutput and Video have finite lifespans determined by expiration timestamps to manage storage resources and ensure data privacy.
- VoiceModel status must be “ready” before it can be used in ProcessingJob to ensure quality and completeness of the cloned voice.

This domain model provides a comprehensive representation of the system’s core business logic and establishes the foundation for all microservices to interact with these entities consistently and reliably.

Chapter 3

System Overview

This chapter provides a comprehensive description of the system’s functionality, architecture, and design. The proposed system implements an end-to-end pipeline for AI-powered video content generation, integrating multiple advanced technologies including deep learning-based speech recognition [?], voice synthesis, and computer vision to deliver a seamless user experience.

3.1 Architectural Design

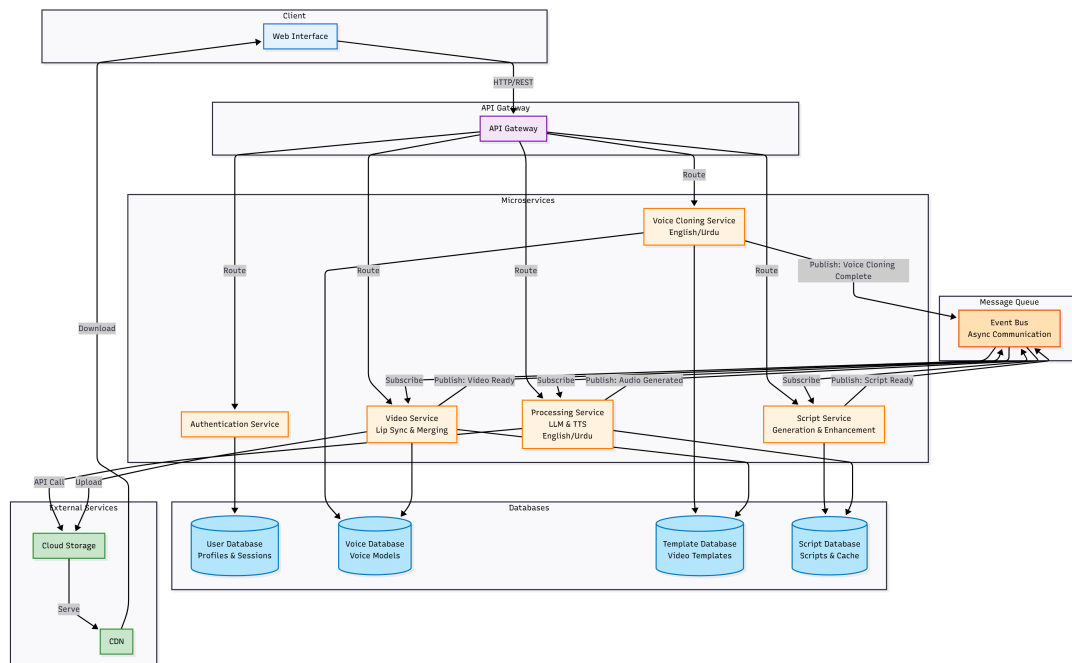


Figure 3.1: High-level Microservices Architecture showing service interactions and communication

The system follows a microservices architecture that decomposes the application into independent, loosely-coupled services. This architecture enables independent development, deployment, and scaling of individual services while maintaining system cohesion through an asynchronous event-driven communication pattern.

The architecture consists of five primary microservices:

Authentication Service: This service handles user identity verification and session management. It manages user login credentials, session creation, and authorization checks. The Authentication Service maintains user profiles and active session data in a dedicated User Database, ensuring secure access control across all other services.

Voice Cloning Service: This service generates personalized voice models for individual users in both English and Urdu languages. It accepts 3-5 audio samples from users as input and creates language-specific voice models through the voice cloning pipeline. The service stores voice models in the Voice Database and user video templates in the Template Database. Upon successful voice model generation, it publishes a “Voice Cloning Complete” event to the Event Bus, notifying dependent services that personalized voice assets are ready for use.

Script Service: This service manages script generation and enhancement through two distinct capabilities. The Script Generation component converts user-provided topics or opinions into draft scripts using language models. The Script Enhancement component refines and improves existing scripts provided by users. Both operations process scripts in the user’s selected language (English or Urdu) and cache results in the Script Database for performance optimization. Upon completion, the service publishes “Script Ready” events to notify dependent services.

Processing Service: This service converts processed scripts into synthetic audio by combining Language Models (LLM) and Text-to-Speech (TTS) engines for both English and Urdu languages. The service retrieves user-specific voice models from the Voice Database and generates high-quality audio output. It maintains processing cache and temporary artifacts in the Script Database for optimization. Upon successful audio generation, it publishes “Audio Generated” events and integrates with Cloud Storage for artifact management.

Video Service: This service generates the final video output by synchronizing generated audio with user video templates. The Lip Synchronization Engine aligns the generated audio with facial movements in the video template. The Video Processing and Merging component combines audio and visual elements to produce the final video. The service retrieves video templates from the Template Database and voice models from the Voice Database, uploads completed videos to Cloud Storage, and publishes “Video Ready” events upon completion.

The services communicate through the following well-defined interfaces:

1. User accesses the system through the Web Interface and authenticates via the Authentication Service
2. For new users: The Voice Cloning Service accepts audio samples and video templates, generates personalized voice models, and publishes “Voice Cloning Complete” events
3. User provides script or topic through the Web Interface to the Script Service
4. The Script Service generates or enhances the script and publishes “Script Ready” events
5. The Processing Service retrieves the script and user voice model, generates synthetic audio, and publishes “Audio Generated” events
6. The Video Service retrieves the generated audio and video template, performs lip synchronization, and completes the video
7. The completed video is uploaded to Cloud Storage and delivered through the CDN
8. User downloads the final video through the Web Interface

Event Bus and Asynchronous Communication: The Event Bus enables asynchronous communication between microservices, preventing blocking operations and enabling loose coupling. Services publish events when they complete operations (“Voice Cloning Complete”, “Script Ready”, “Audio Generated”, “Video Ready”), and dependent services subscribe to these events. This pattern allows services to operate independently while maintaining coordination through event-driven workflows.

Data Layer: Each microservice maintains its own database following the database-per-service pattern. The User Database stores authentication credentials and session information. The Voice Database persists cloned voice models for both languages. The Template Database stores user video templates. The Script Database maintains scripts and provides processing cache for optimization. This separation ensures data isolation and enables independent database optimization for each service.

External Services: Cloud Storage provides temporary and persistent storage for processing artifacts and generated videos. The Content Delivery Network (CDN) ensures efficient global distribution of generated videos to end-users with optimized performance and reduced latency.

This microservices decomposition enables parallel development of independent services, facilitates horizontal scaling of resource-intensive operations (voice cloning, video processing), supports future extensions such as additional language support or advanced avatar features, and provides fault isolation ensuring that failures in individual services do not cascade to the entire system.

3.2 Design Models

This section presents detailed design models that capture the system's behavior, data flow, and component interactions.

3.2.1 Data Flow Diagram - Level 0 (Context Diagram)

The context diagram provides a high-level view of the system and its interactions with external entities.

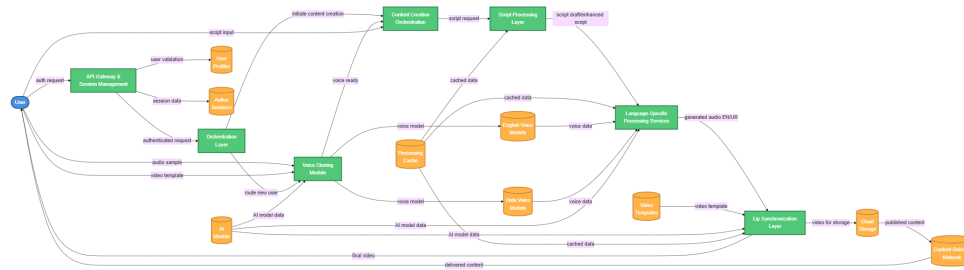


Figure 3.2: Data Flow Diagram Level 0 - Context Diagram

The Level 0 DFD shows the system as a single process that accepts audio/video inputs, user credentials, and template selections from users, and produces refined transcripts, synthetic speech, and lip-synced videos as outputs.

3.2.2 Data Flow Diagram - Level 1

The Level 1 diagram decomposes the system into major processes and data stores.

The Level 1 DFD illustrates:

- **Process 1: User Authentication & Management** - Validates users and manages profiles
- **Process 2: Speech-to-Text Processing** - Transcribes audio to text
- **Process 3: Text Refinement** - Enhances transcript quality using language models

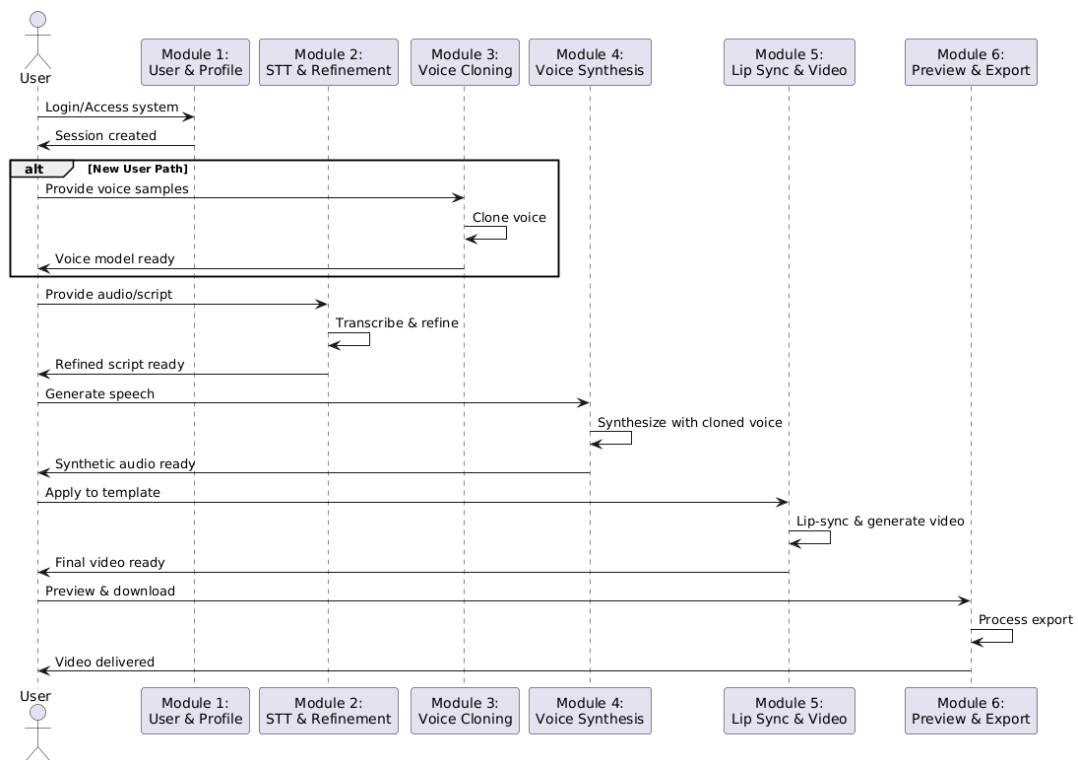


Figure 3.4: Module 0 - High-level end-to-end workflow showing interaction between all system modules

3.2.3.2 Module 1: User & Profile Management

Module 1 handles all user-related operations including authentication, session creation, and profile management. The sequence diagram illustrates how a user logs into the system, credentials are validated against the user database, sessions are created and stored, and user state is initialized. This module serves as the entry point for all users and ensures secure access to the system. For new users, it triggers the voice cloning process, while returning users can immediately access existing voice models and proceed with content creation.

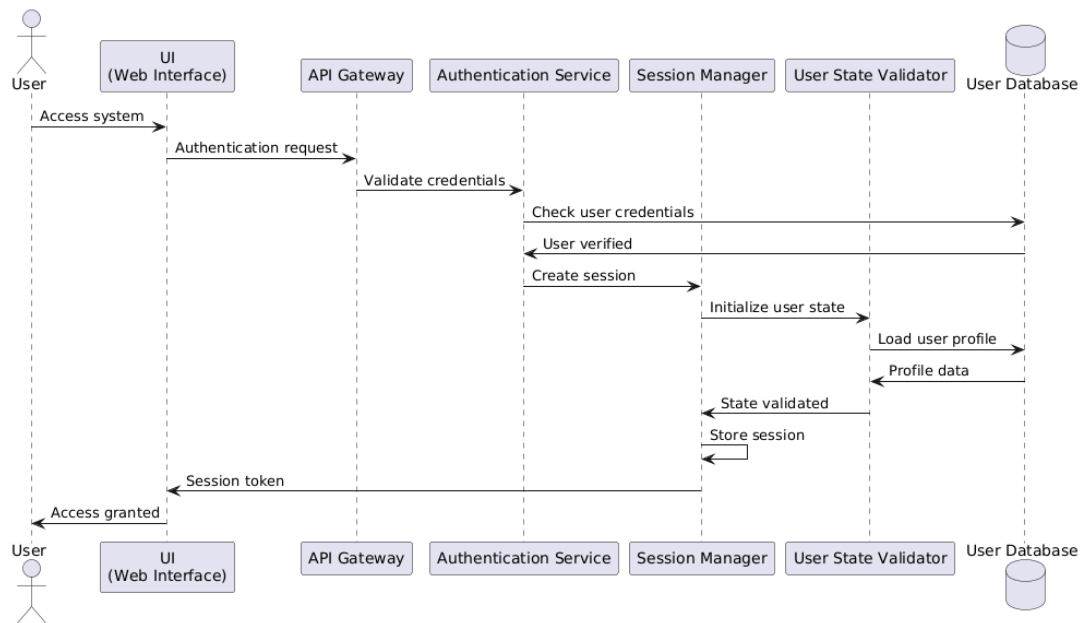


Figure 3.5: Module 1 - User authentication, session management, and profile initialization

3.2.3.3 Module 2: Speech-to-Text Transcription & Text Refinement

Module 2 is responsible for converting audio input into refined text. The sequence begins with the user uploading an audio file, which is routed based on the detected language (English or Urdu). The speech-to-text engine transcribes the audio into a raw transcript, which is then processed by the appropriate language model (LLMEN for English or LLMUR for Urdu) to enhance quality, correct errors, and improve grammatical structure. The refined transcript is cached for future reference and returned to the user for review and approval. This module ensures high-quality text input for subsequent processing stages.

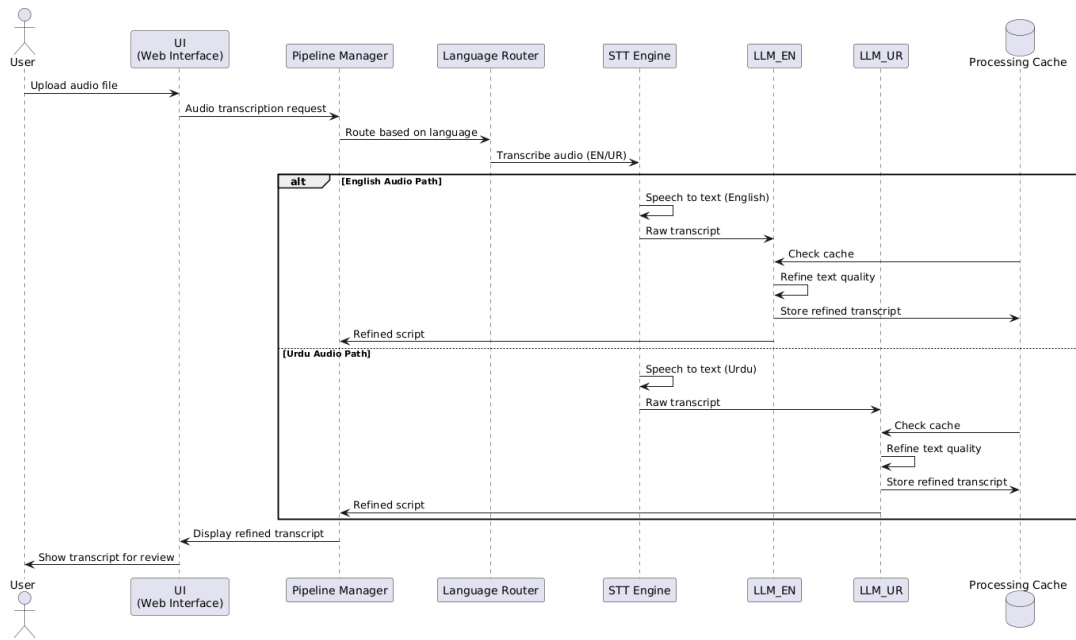


Figure 3.6: Module 2 - Audio transcription and language model-based text enhancement

3.2.3.4 Module 3: Adaptive Voice Cloning

Module 3 implements the adaptive voice cloning mechanism that enables personalized voice synthesis. For new users, this module trains a voice model using provided audio samples in the appropriate language (English or Urdu). The trained model is stored in the voice model repository and associated with the user's profile. For returning users, the system retrieves the existing voice model from storage, eliminating the need for re-training. This adaptive approach ensures that output quality improves over time as users interact with the system repeatedly, creating a personalized experience tailored to each individual's voice characteristics.

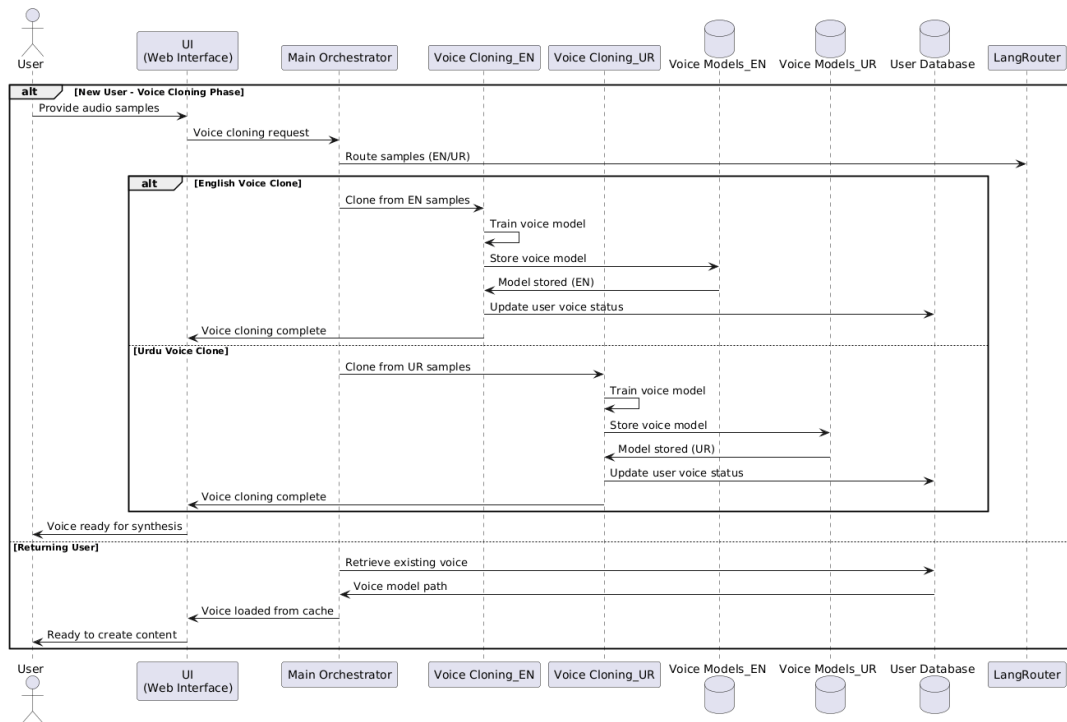


Figure 3.7: Module 3 - Voice model training and adaptive voice cloning for personalized synthesis

3.2.3.5 Module 4: Voice Synthesis & Text-to-Speech

Module 4 converts the refined text into synthetic speech using the user's cloned voice model. The refined script is routed to the appropriate TTS engine based on language selection. The engine retrieves the user's previously trained voice model and generates speech that closely mimics the user's natural voice. Processing caches are consulted to avoid redundant computations, and generated audio files are stored for subsequent use. This module produces high-quality synthetic audio that sounds natural and personalized to the individual user.

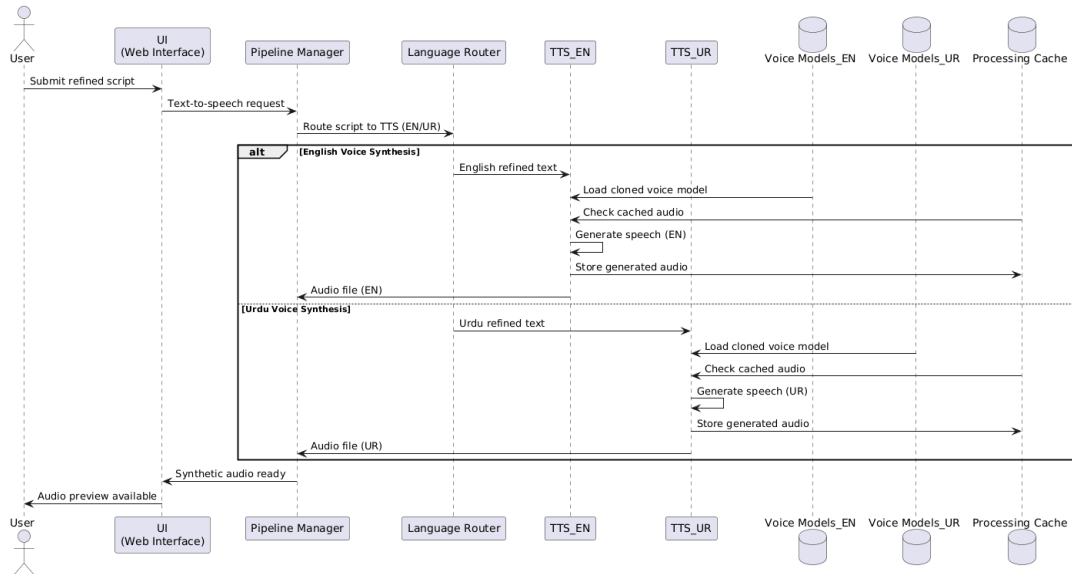


Figure 3.8: Module 4 - Text-to-speech conversion using cloned voice models

3.2.3.6 Module 5: Lip Synchronization & Video Generation

Module 5 combines the synthetic audio with the selected video template to produce the final lip-synced video. The sequence diagram illustrates how the Lip Sync Engine analyzes the audio frames, detects mouth movements and phoneme patterns, and applies the Wav2Lip algorithm to animate the video template's facial movements in synchronization with the generated speech. The Video Processing and Merging component then composites the synchronized video frames with the audio track, applies quality settings, and stores the final rendered output. This module is critical for creating professional-looking videos where the speaker's mouth movements match the generated audio.

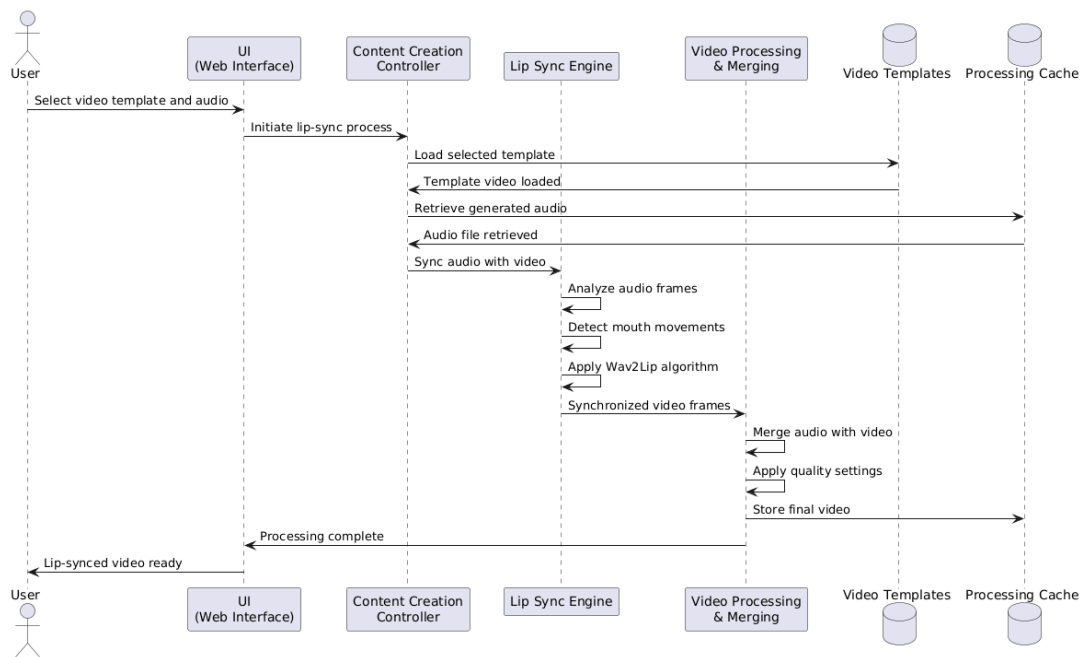


Figure 3.9: Module 5 - Lip-sync alignment and final video generation

3.2.3.7 Module 6: Preview, Export & Content Delivery

Module 6 handles the final stages of content delivery, allowing users to preview their generated video before export and manage the download process. The Preview Manager retrieves the final video from cache and displays it in an interactive player, enabling users to review quality and make any necessary approvals. Once confirmed, the Download Manager prepares the video for export and uploads it to the Content Delivery Network (CDN) for distribution. The CDN replicates the content across multiple geographic servers, ensuring fast and reliable delivery to end users. This module completes the content generation pipeline by making the final product accessible to the user.

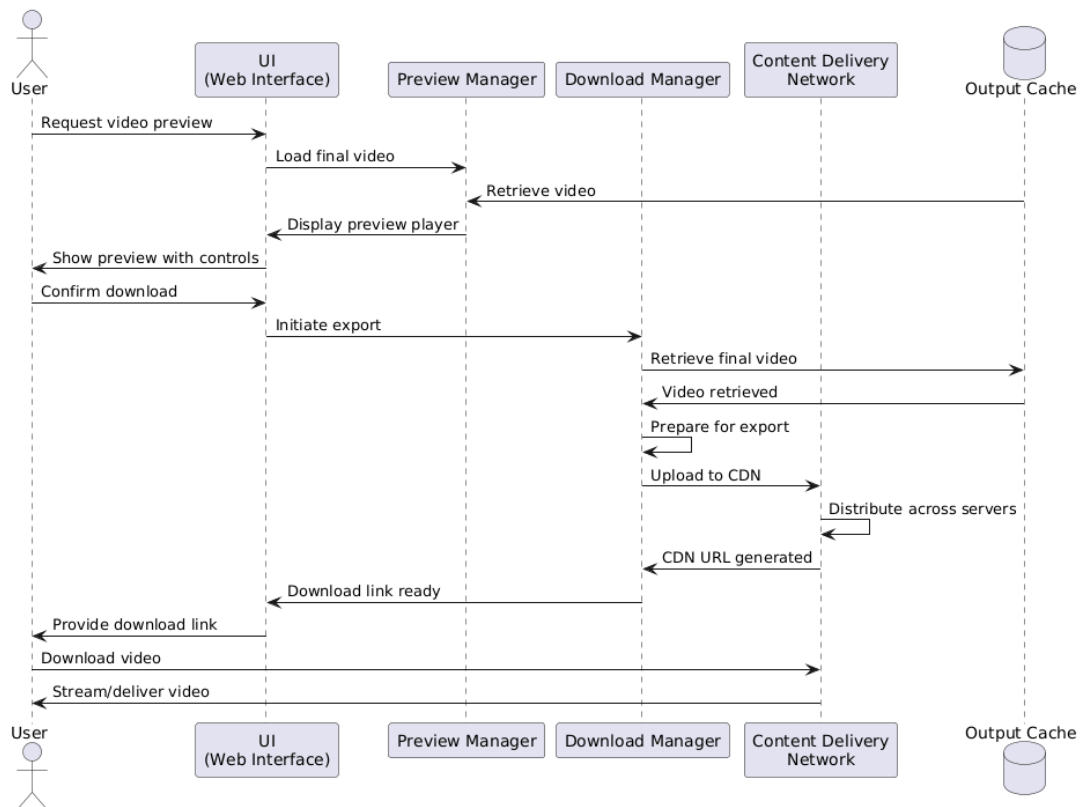


Figure 3.10: Module 6 - Video preview, download management, and CDN distribution

3.2.4 Database Schema

The system uses a relational database to manage structured data with the following primary entities:

Users Table:

- `user_id` (Primary Key): Unique identifier

- email: User email address (unique)
- password_hash: Encrypted password
- created_at, updated_at: Timestamps
- language_preference: Default language (Urdu/English)
- consent_voice_storage: Boolean flag for consent

Audio_Samples Table:

- sample_id (Primary Key): Unique identifier
- user_id (Foreign Key): Reference to Users
- file_path: Storage location of audio file
- duration: Audio length in seconds
- language: Detected language
- upload_timestamp: Upload date and time
- transcription: Generated text transcript

Voice_Models Table:

- model_id (Primary Key): Unique identifier
- user_id (Foreign Key): Reference to Users
- model_path: Storage location of trained model
- training_samples: Number of samples used for training
- quality_score: Evaluated quality metric (MOS)
- version: Model version number for tracking improvements
- created_at: Training timestamp

Text_Contents Table:

- content_id (Primary Key): Unique identifier
- user_id (Foreign Key): Reference to Users

- `original_transcript`: Raw STT output
- `refined_script`: Language model enhanced text
- `language`: Content language
- `word_count`: Number of words
- `created_at`: Creation timestamp

Video_Templates Table:

- `template_id` (Primary Key): Unique identifier
- `template_name`: Descriptive name
- `file_path`: Storage location of template video
- `resolution`: Video resolution (e.g., 1920x1080)
- `duration`: Template duration in seconds
- `thumbnail_path`: Preview image location
- `is_public`: Boolean flag for shared templates

Generated_Videos Table:

- `video_id` (Primary Key): Unique identifier
- `user_id` (Foreign Key): Reference to Users
- `template_id` (Foreign Key): Reference to Video_Templates
- `voice_model_id` (Foreign Key): Reference to Voice_Models
- `content_id` (Foreign Key): Reference to Text_Contents
- `output_path`: Storage location of final video
- `rendering_settings`: JSON object with quality parameters
- `processing_time`: Time taken to generate video
- `created_at`: Generation timestamp

3.2.5 File Storage Organization

Media files and ML models are stored in a hierarchical file system structure:

```

/storage/
  /users/
    /{user_id}/
      /audio/          # Raw audio uploads
      /videos/         # Generated videos
      /voice_models/   # User-specific voice models
  /templates/
    /public/           # Shared video templates
    /private/          # User-uploaded templates
  /cache/
    /temp/             # Temporary processing files
    /thumbnails/       # Preview images

```

Files are named using UUIDs to prevent conflicts and ensure uniqueness. Metadata is stored in the database with references to file paths.

3.2.6 Data Processing Pipeline

Data flows through the system in the following sequence:

1. **Input Stage:** Raw audio/video uploaded and stored; metadata created in database
2. **Transcription Stage:** Audio processed by STT model; transcript stored in Text_Contents
3. **Refinement Stage:** Transcript enhanced by language model; updated in database
4. **Voice Synthesis Stage:** Refined text converted to speech using cloned voice model
5. **Lip Sync Stage:** Generated audio synchronized with video template
6. **Output Stage:** Final video rendered, stored, and database record created

All processing stages implement error handling and rollback mechanisms to ensure data consistency. Processing state is tracked through status fields in database records, enabling resume functionality for interrupted operations.

3.3 Conclusion

This proposal presents a unified, bilingual (Urdu/English) pipeline for generating polished, single-speaker videos from minimal inputs. By integrating speech-to-text transcription, language model-based text refinement, adaptive voice cloning, and lip synchronization technologies, the system significantly reduces production time, eliminates the need for tool switching, and minimizes the technical expertise required for professional content creation. The modular architecture ensures maintainability and extensibility while delivering high-quality outputs that maintain user control throughout the generation process.

The system addresses a critical gap in the content creation ecosystem by providing native support for Urdu alongside English, recognizing the linguistic needs of millions of content creators in South Asia. The adaptive nature of the voice cloning component ensures that output quality improves with continued use, creating a personalized experience that becomes more valuable over time. By consolidating multiple production steps into a single workflow, the system empowers creators, educators, influencers, and businesses to produce professional-quality videos without substantial investments in equipment, software, or specialized skills.

The technical foundation established through this project—combining state-of-the-art speech processing, natural language understanding, voice synthesis, and computer vision—demonstrates the viability of end-to-end AI-powered content generation systems. The focus on usability, performance, and quality ensures that the system meets real-world production demands while remaining accessible to non-technical users.

3.4 Future Work

While the initial system focuses on single-speaker content with template-based video generation, several promising directions for future enhancement have been identified:

Multi-Speaker Dialogue Support: Extending the system to handle conversations and dialogues involving multiple speakers would significantly broaden its applicability. This enhancement would require speaker diarization, individual voice model management for each speaker, and coordinated lip synchronization across multiple faces in the video frame.

Advanced Avatar Animation: Moving beyond template-based lip synchronization to full avatar generation and animation would enable users to create content without requiring pre-recorded video templates. This could include 3D avatar modeling, body language synthesis, and emotional expression control synchronized with speech prosody.

Improved Urdu ASR Accuracy: While current Urdu speech recognition capabilities are functional, there remains significant room for improvement, particularly for regional dialects and accents. Future work could involve training specialized models on larger, more diverse Urdu speech corpora and implementing accent-adaptive recognition techniques.

Latency Reduction for Real-Time Feedback: Optimizing the processing pipeline to achieve near-real-time performance would enable interactive content creation workflows. This could involve model quantization, efficient caching strategies, and GPU acceleration for compute-intensive operations.

Broader Device Support: Expanding platform support to include mobile applications (iOS and Android) would make the system accessible to users who primarily create content on smartphones and tablets. This would require developing optimized inference pipelines suitable for resource-constrained mobile environments.

Low-Resource Urdu TTS Enhancement: Research into robust, low-resource Urdu text-to-speech synthesis would improve voice quality and naturalness, particularly for users with limited voice samples. Transfer learning from high-resource languages and cross-lingual voice adaptation techniques show promise in this direction.

Long-Form Prosody Control: Extending the system to handle longer content (beyond five minutes) while maintaining natural prosody, appropriate pacing, and emotional expressiveness would require sophisticated prosody modeling and contextual speech synthesis capabilities.

Collaborative Features: Introducing collaborative workflows where multiple users can contribute to content creation, with role-based access control and version management, would position the system for team-based production environments.

Automated Quality Assessment: Implementing objective quality metrics and automated testing for output videos would reduce the need for manual review and ensure consistent quality standards across all generated content.

These future enhancements would further strengthen the system's value proposition and expand its applicability across diverse use cases and user segments, ultimately contributing to democratizing high-quality content creation in underserved linguistic markets.

Chapter 4

Implementation and Testing

This chapter provides a comprehensive overview of the implementation and testing approaches employed in developing the Urdu AI Video Creator platform. The system integrates modern authentication, AI-powered script generation, and content creation workflows to deliver a seamless user experience for creating multilingual video content.

4.1 General Description

The Urdu AI Video Creator is a Next.js-based web application designed to enable content creators to generate AI-powered videos with support for Urdu and English languages. The platform features enterprise-grade authentication, intelligent script generation, voice cloning capabilities, and lip-sync technology.

4.1.1 System Context

The system addresses the growing demand for multilingual content creation tools, particularly for Urdu language content. Traditional video creation workflows are time-consuming and require significant technical expertise. This platform automates script generation, voice synthesis, and video production through AI integration.

4.1.2 Technology Stack

- **Frontend:** Next.js 14 with TypeScript, Tailwind CSS, Radix UI components
- **Backend:** Node.js with Express.js, TypeScript
- **Authentication:** Clerk for enterprise-grade authentication and session management

- **Database:** Supabase (PostgreSQL) with Row Level Security
- **AI Services:** OpenRouter API for LLM integration (GPT-4), OpenAI Whisper for transcription
- **Text-to-Speech:** Coqui XTTS v2 for voice cloning and synthesis (English: pre-trained, Urdu: custom fine-tuned)
- **Additional Libraries:** Framer Motion for animations, PyTorch for model inference

4.1.3 Current Implementation Status

The current iteration includes:

- Complete authentication system with user management
- Multi-stage content creation workflow
- AI-powered script generation and refinement
- Iterative feedback-based script improvement
- Draft management and version control
- Protected dashboard with user-specific content access

4.2 Algorithm Design

This section details the core algorithms and workflows implemented in the system.

4.2.1 User Authentication and Synchronization

The authentication flow ensures secure user access and data synchronization between Clerk and Supabase.

Algorithm 1 User Authentication and Synchronization

```

1: Input: User credentials (email, password) or OAuth provider
2: Output: Authenticated session with synchronized user data
3: procedure AUTHENTICATEUSER(credentials)
4:   user ← Clerk.authenticate(credentials)
5:   if authentication successful then
6:     token ← generateJWT(user.clerkId)
7:     session ← createSession(token)
8:     SYNCUSERTOSUPABASE(user.clerkId,      user.email,      user.firstName,
        user.lastName)
9:     return session
10:  else
11:    return error("Authentication failed")
12:  end if
13: end procedure
14: procedure SYNCUSERTOSUPABASE(clerkId, email, firstName, lastName)
15:   token ← getCurrentSessionToken()
16:   headers ← {"Authorization": "Bearer " + token}
17:   body ← {clerkId, email, firstName, lastName}
18:   response ← POST("/api/users/sync", headers, body)
19:   if response.success then
20:     Supabase.upsert("users", {clerk_id: clerkId, email, first_name: firstName,
        last_name: lastName})
21:   end if
22: end procedure
23: procedure VERIFYREQUEST(request)
24:   token ← extractToken(request.headers["Authorization"])
25:   session ← Clerk.verifyToken(token)
26:   if session is valid then
27:     request.auth ← {userId: session.sub}
28:     continue to protected route
29:   else
30:     return 401 Unauthorized
31:   end if
32: end procedure

```

API and Version	Purpose of Usage	API Endpoint/Function/Class Used
Clerk (v6.35.6)	User signup, login, session management, JWT token generation	ClerkProvider, useAuth(), clerkClient.verifyToken(), clerkMiddleware()
Supabase (v2.86.2)	Store user data, scripts, versions, and metadata with RLS	createClient(), from('users'), from('scripts'), upsert(), select()
OpenRouter API	Generate and refine scripts using AI models	POST https://openrouter.ai/api/v1/chat/comp
OpenAI API (v5.20.3)	Audio transcription and language detection	openai.audio.transcriptions.create(), whisper-1 model
Coqui XTTS v2	Zero-shot and few-shot voice cloning for English; custom fine-tuned model for Urdu language support	TTS.api.TTS(), tts.tts_to_file(), xtts_v2 model
Next.js (v14.2.32)	Server-side rendering, API routes, routing, and middleware	next/navigation, middleware.ts, API Routes
Radix UI (Multiple)	Build accessible forms, dialogs, dropdowns, and interactive elements	@radix-ui/react-dialog, @radix-ui/react-select, @radix-ui/react-checkbox
Framer Motion (latest)	Smooth transitions and animations in UI	motion.div, AnimatePresence, variants

4.2.2 Iterative Script Refinement with Feedback

Algorithm 2 Feedback-Based Script Refinement

```

1: Input: scriptId, currentScript, feedback, language, duration, pacing
2: Output: Refined script with incremented version number
3: procedure REFINEWITHFEEDBACK(scriptId, currentScript, feedback, language, du-
   ration, pacing)
4:   userId ← getCurrentUserId()
5:   existingScript ← FETCHSCRIPT(scriptId, userId)
6:   if existingScript is null then
7:     return error("Script not found")
8:   end if
9:   currentVersion ← length(existingScript.versions)
10:  prompt ← BUILDFEEDBACKPROMPT(language, duration, pacing, feedback, cur-
   rentVersion)
11:  prompt += "\n\nCurrent script:\n" + currentScript
12:  messages ← [{"role": "user", "content": prompt}]
13:  refinedText ← CALLLLM(messages, maxTokens=4000)
14:  metadata ← CALCULATEMETADATA(refinedText, pacing)
15:  newVersion ← {
16:    versionNumber: currentVersion + 1,
17:    content: refinedText,
18:    feedback: feedback,
19:    createdAt: getCurrentTimestamp()
20:  }
21:  existingScript.versions.append(newVersion)
22:  existingScript.content ← refinedText
23:  existingScript.metadata ← metadata
24:  UPDATEDATABASE(scriptId, existingScript)
25:  return {scriptId, content: refinedText, metadata, version: currentVersion + 1}
26: end procedure
27: procedure BUILDFEEDBACKPROMPT(language, duration, pacing, feedback, ver-
   sion)
28:  prompt ← "You are refining a script based on user feedback."
29:  prompt += "This is version " + (version + 1) + " of the script."
30:  prompt += "Language: " + language
31:  prompt += "Target duration: " + duration + " seconds"
32:  prompt += "Pacing: " + pacing
33:  prompt += "\n\nUser feedback: " + feedback
34:  prompt += "\n\nApply the feedback to improve the script while maintaining its
   core message."
35:  return prompt
36: end procedure

```

4.2.3 Script Refinement (Simple and Custom)

Algorithm 3 Script Refinement Process

```

1: Input: originalScript, refinementType, customInstructions (optional), language, du-
   ration, pacing
2: Output: Refined script
3: procedure REFINEScript(originalScript, refinementType, customInstructions, lan-
   guage, duration, pacing)
4:   userId ← getCurrentUserId()
5:   if refinementType == “simple” then
6:     prompt ← GETSIMPLEREFINEMENTPROMPT(language, duration, pacing)
7:   else if refinementType == “custom” then
8:     if customInstructions is empty then
9:       return error(“Custom instructions required”)
10:    end if
11:    prompt ← GETCUSTOMREFINEMENTPROMPT(language, duration, pacing,
      customInstructions)
12:   else
13:     return error(“Invalid refinement type”)
14:   end if
15:   prompt += “\n\nOriginal script:\n” + originalScript
16:   messages ← [{“role”: “user”, “content”: prompt}]
17:   refinedText ← CALLLLM(messages, maxTokens=4000)
18:   metadata ← CALCULATEMETADATA(refinedText, pacing)
19:   scriptId ← generateUUID()
20:   script ← {scriptId, userId, content: refinedText, method: “refinement”, metadata}
21:   SAVEDATABASE(script)
22:   return {scriptId, content: refinedText, metadata}
23: end procedure

```

4.2.4 Voice Cloning with XTTS v2

The text-to-speech synthesis uses Coqui XTTS v2, a state-of-the-art voice cloning model supporting both zero-shot and few-shot approaches.

Algorithm 4 Voice Cloning and Speech Synthesis

```

1: Input: script text, language, speaker_wav (reference audio for cloning)
2: Output: Synthesized speech audio file
3: procedure SYNTHESIZESPEECH(text, language, speaker_wav)
4:   device ← “cuda” if GPU available else “cpu”
5:   if language == “English” then
6:     modelName ← “tts_models/multilingual/multi-dataset/xtts_v2”      ▷
     Pre-trained model
7:   else if language == “Urdu” then
8:     modelName ← “custom_models/urdu/xtts_v2_finetuned”              ▷ Custom
     fine-tuned model
9:   else
10:    return error(“Unsupported language”)
11:  end if
12:  tts ← INITIALIZE_TTS(modelName, device)
13:  VALIDATEREFERENCEAUDIO(speaker_wav)
14:  audioWav ← tts.tts(
15:    text: text,
16:    speaker_wav: speaker_wav,
17:    language: language
18:  )
19:  outputPath ← generateOutputPath()
20:  tts.tts_to_file(
21:    text: text,
22:    speaker_wav: speaker_wav,
23:    language: language,
24:    file_path: outputPath
25:  )
26:  return outputPath
27: end procedure
28: procedure VALIDATEREFERENCEAUDIO(speaker_wav)
29:   if not fileExists(speaker_wav) then
30:     return error(“Reference audio file not found”)
31:   end if
32:   duration ← getAudioDuration(speaker_wav)
33:   if duration < 6 or duration > 30 then
34:     return error(“Reference audio should be 6-30 seconds”)
35:   end if
36: end procedure

```

4.2.5 Custom XTTS v2 Training for Urdu Language

Since no existing TTS model supports both Urdu language and voice cloning, a custom fine-tuning pipeline was developed to train XTTS v2 on Urdu language data.

4.2.5.1 Dataset Collection and Preprocessing

Multiple open-source Urdu speech datasets were collected and preprocessed:

- **NLP-Story2Audio Dataset:** Urdu audio transcripts from GitHub (m01omer/NLP-Story2Audio)
- **Common Voice Urdu:** Preprocessed version by UmarRamzan (common-voice-urdu-processed)
- **Mendeley Urdu Speech Dataset:** Academic dataset (jcpfjnk5c2/3)
- **Jalandhary STT-TTS Dataset:** Kaggle dataset (mirfan899/jalandhary-stt-tts)

Algorithm 5 Urdu Dataset Preprocessing Pipeline

```

1: Input: Raw audio files and transcriptions from multiple sources
2: Output: Preprocessed dataset ready for training
3: procedure PREPROCESSURDUDATASET(rawDatasets)
4:   combinedData  $\leftarrow$  []
5:   for each dataset in rawDatasets do
6:     audioFiles  $\leftarrow$  dataset.getAudioFiles()
7:     transcripts  $\leftarrow$  dataset.getTranscripts()
8:     for each (audio, transcript) pair do
9:       VALIDATEAUDIOQUALITY(audio)
10:      NORMALIZETRSCRIPT(transcript)
11:      RESAMPLEAUDIO(audio, targetRate=22050)
12:      NORMALIZEAUDIOLEVEL(audio)
13:      combinedData.append({ audio, transcript })
14:    end for
15:  end for
16:  trainSet, valSet  $\leftarrow$  SPLITDATASET(combinedData, ratio=0.9)
17:  return trainSet, valSet
18: end procedure
19: procedure VALIDATEAUDIOQUALITY(audio)
20:   sampleRate  $\leftarrow$  getAudioSampleRate(audio)
21:   duration  $\leftarrow$  getAudioDuration(audio)
22:   snr  $\leftarrow$  calculateSNR(audio)
23:   if duration < 1 or duration > 20 then
24:     skip this sample
25:   end if
26:   if snr < 20 then ▷ Signal-to-noise ratio threshold
27:     skip this sample
28:   end if
29: end procedure
30: procedure NORMALIZETRSCRIPT(transcript)
31:   transcript  $\leftarrow$  removeExtraWhitespace(transcript)
32:   transcript  $\leftarrow$  normalizeUrduCharacters(transcript)
33:   transcript  $\leftarrow$  removeDiacritics(transcript) ▷ Optional
34:   return transcript
35: end procedure

```

4.2.5.2 Vocabulary Extension with BPE

To handle Urdu script properly, the vocabulary was extended using Byte Pair Encoding (BPE).

Algorithm 6 Vocabulary Extension for Urdu Script

```

1: Input: Urdu text corpus from preprocessed transcripts
2: Output: Extended vocabulary tokenizer
3: procedure EXTENDVOCABULARY(urduCorpus, baseTokenizer)
4:   allText  $\leftarrow$  concatenate all Urdu transcripts
5:   urduCharacters  $\leftarrow$  extractUniqueCharacters(allText)
6:   urduSubwords  $\leftarrow$  TRAINBPE(allText, vocabSize=5000)
7:   newVocab  $\leftarrow$  baseTokenizer.vocab
8:   for each token in urduSubwords do
9:     if token not in newVocab then
10:       newVocab.add(token)
11:     end if
12:   end for
13:   extendedTokenizer  $\leftarrow$  updateTokenizer(baseTokenizer, newVocab)
14:   SAVEDTOKENIZER(extendedTokenizer, "urdu_xtts_tokenizer.json")
15:   return extendedTokenizer
16: end procedure
17: procedure TRAINBPE(corpus, vocabSize)
18:   tokenizer  $\leftarrow$  ByteLevelBPETokenizer()
19:   tokenizer.train(
20:     files: [corpus],
21:     vocab_size: vocabSize,
22:     min_frequency: 2,
23:     special_tokens: ["<pad>", "<sos>", "<eos>", "<unk>"]
24:   )
25:   return tokenizer.get_vocab()
26: end procedure

```

4.2.5.3 XTTS v2 Fine-tuning Pipeline

The fine-tuning process adapts the pre-trained XTTS v2 model to Urdu language while preserving voice cloning capabilities.

Algorithm 7 XTTS v2 Fine-tuning for Urdu

```

1: Input: Pre-trained XTTS v2 model, preprocessed Urdu dataset, extended tokenizer
2: Output: Fine-tuned Urdu XTTS v2 model
3: procedure FINETUNEXTTS(baseModel, urduDataset, tokenizer)
4:   config  $\leftarrow$  loadXTTSConfig("xtts_v2_config.json")
5:   config.vocab_file  $\leftarrow$  "urdu_xtts_tokenizer.json"
6:   config.language  $\leftarrow$  "ur"
7:   model  $\leftarrow$  LOADPRETRAINEDMODEL(baseModel)
8:   model.tokenizer  $\leftarrow$  tokenizer
9:    $\triangleright$  Freeze encoder layers, train only GPT and decoder
10:  for layer in model.speaker_encoder do
11:    layer.requires_grad  $\leftarrow$  False
12:  end for
13:  optimizer  $\leftarrow$  AdamW(
14:    params: model.trainable_parameters(),
15:    lr: 5e-6,
16:    weight_decay: 1e-5
17:  )
18:  scheduler  $\leftarrow$  CosineAnnealingLR(optimizer, T_max: epochs)
19:  for epoch in range(50) do
20:    for batch in urduDataset.train_loader do
21:      audio, text, speaker_embedding  $\leftarrow$  batch
22:      loss  $\leftarrow$  model.compute_loss(audio, text, speaker_embedding)
23:      loss.backward()
24:      optimizer.step()
25:      optimizer.zero_grad()
26:      if step modulo 100 == 0 then
27:        LOGMETRICS(epoch, step, loss)
28:        VALIDATEMODEL(model, urduDataset.val_loader)
29:      end if
30:    end for
31:    scheduler.step()
32:    SAVECHECKPOINT(model, epoch)
33:  end for
34:  return model
35: end procedure
36: procedure VALIDATEMODEL(model, valLoader)
37:   totalLoss  $\leftarrow$  0
38:   for batch in valLoader do
39:     audio, text, speaker_embedding  $\leftarrow$  batch
40:     loss  $\leftarrow$  model.compute_loss(audio, text, speaker_embedding)
41:     totalLoss += loss
42:   end for
43:   avgLoss  $\leftarrow$  totalLoss / length(valLoader)
44:   log validation loss: avgLoss
45: end procedure

```

4.2.5.4 Training Environment Challenges

Due to Coqui TTS company shutdown in 2024, the library lacks support for latest Python versions and dependencies. The training was conducted on local systems with compatible environments:

- **Python Version:** 3.9.x (compatibility requirement)
- **PyTorch:** 1.13.x with CUDA 11.7
- **Training Hardware:** NVIDIA GPU with minimum 16GB VRAM
- **Training Duration:** Approximately 48-72 hours on consumer-grade GPU

4.3 External APIs/SDKs

The following table describes third-party APIs and SDKs integrated into the system:

API and Version	Purpose of Usage	API Endpoint/Function/Class Used
Clerk (v6.35.6)	User signup, login, session management, JWT token generation	ClerkProvider, useAuth(), clerkClient.verifyToken(), clerkMiddleware()
Supabase (v2.86.2)	Store user data, scripts, versions, and metadata with RLS	createClient(), from('users'), from('scripts'), upsert(), select()
OpenRouter API	Generate and refine scripts using AI models	POST https://openrouter.ai/api/v1
OpenAI API (v5.20.3)	Audio transcription and language detection	openai.audio.transcriptions.create(), whisper-1 model
Coqui XTTS v2	Zero-shot and few-shot voice cloning for English; custom fine-tuned model for Urdu language support	TTS.api.TTS(), tts.tts_to_file(), xtts_v2 model
Next.js (v14.2.32)	Server-side rendering, API routes, routing, and middleware	next/navigation, middleware.ts, API Routes
Radix UI (Multiple)	Build accessible forms, dialogs, dropdowns, and interactive elements	@radix-ui/react-dialog, @radix-ui/react-select, @radix-ui/react-checkbox
Framer Motion (latest)	Smooth transitions and animations in UI	motion.div, AnimatePresence, variants

4.4 Testing Details

Testing ensures that the system functions correctly, securely, and efficiently. This section covers the testing methodologies employed during development.

4.4.1 Unit Testing

Unit tests validate individual components and functions in isolation. Each test targets a specific function to ensure correctness.

4.4.1.1 Authentication Middleware Test

Test Case: Verify that the authentication middleware correctly validates JWT tokens and rejects unauthorized requests.

```
// Test File: server/src/middleware/__tests__/auth.test.ts

import { requireAuth } from '../auth';
import { Request, Response, NextFunction } from 'express';
import { clerkClient } from '@clerk/clerk-sdk-node';

jest.mock('@clerk/clerk-sdk-node');

describe('Authentication Middleware', () => {
  let mockRequest: Partial<Request>;
  let mockResponse: Partial<Response>;
  let nextFunction: NextFunction;

  beforeEach(() => {
    mockRequest = {
      headers: {},
    };
    mockResponse = {
      status: jest.fn().mockReturnThis(),
      json: jest.fn(),
    };
    nextFunction = jest.fn();
  });
```

```
test('should reject requests without Authorization header',
  async () => {
    await requireAuth(mockRequest as Request,
      mockResponse as Response,
      nextFunction);

    expect(mockResponse.status).toHaveBeenCalledWith(401);
    expect(mockResponse.json).toHaveBeenCalledWith({
      error: 'Missing or invalid Authorization header'
    });
    expect(nextFunction).not.toHaveBeenCalled();
  });

test('should reject requests with invalid Bearer token',
  async () => {
    mockRequest.headers = { authorization: 'Bearer invalid_token' };

    (clerkClient.verifyToken as jest.Mock).mockRejectedValue(
      new Error('Invalid token')
    );

    await requireAuth(mockRequest as Request,
      mockResponse as Response,
      nextFunction);

    expect(mockResponse.status).toHaveBeenCalledWith(401);
    expect(mockResponse.json).toHaveBeenCalledWith({
      error: 'Authentication failed'
    });
  });

test('should attach userId to request for valid tokens',
  async () => {
    const validToken = 'valid_jwt_token';
    mockRequest.headers = {
      authorization: `Bearer ${validToken}`
    };

    (clerkClient.verifyToken as jest.Mock).mockResolvedValue({
      sub: 'user_123456'
    });
  });
```

```

});

await requireAuth(mockRequest as Request,
                  mockResponse as Response,
                  nextFunction);

expect(mockRequest.auth).toEqual({
  userId: 'user_123456',
  getToken: expect.any(Function)
});
expect(nextFunction).toHaveBeenCalled();
});
});

```

Expected Results:

- Requests without authorization headers return 401 status
- Invalid tokens are rejected with appropriate error messages
- Valid tokens attach user ID to request and proceed to next middleware

4.4.1.2 Script Generation Metadata Calculation Test

Test Case: Validate that word count and duration calculations are accurate for different pacing settings.

```

// Test File:
// server/src/controllers/__tests__/contentController.test.ts

import { calculateMetadata } from '../contentController';

describe('Script Metadata Calculation', () => {

  test('should calculate correct word count', () => {
    const script = 'This is a sample script with ten words here.';
    const { wordCount } = calculateMetadata
      <parameter name="content">(script, 'medium');

    expect(wordCount).toBe(10);
  });

```

```
test('should calculate duration for slow pacing (120 WPM)',
  () => {
    const script = 'word '.repeat(240); // 240 words
    const { estimatedDuration } = calculateMetadata(script, 'slow');

    expect(estimatedDuration).toBe(2); // 240 / 120 = 2 minutes
  });

test('should calculate duration for medium pacing (140 WPM)',
  () => {
    const script = 'word '.repeat(280); // 280 words
    const { estimatedDuration } = calculateMetadata(script,
                                                    'medium');

    expect(estimatedDuration).toBe(2); // 280 / 140 = 2 minutes
  });

test('should calculate duration for fast pacing (160 WPM)',
  () => {
    const script = 'word '.repeat(320); // 320 words
    const { estimatedDuration } = calculateMetadata(script, 'fast');

    expect(estimatedDuration).toBe(2); // 320 / 160 = 2 minutes
  });

test('should handle empty scripts', () => {
  const script = '';
  const { wordCount, estimatedDuration } =
    calculateMetadata(script, 'medium');

  expect(wordCount).toBe(0);
  expect(estimatedDuration).toBe(0);
});
```

Expected Results:

- Word count accurately reflects number of words in script
- Duration calculations adjust correctly based on pacing (slow: 120 WPM, medium:

140 WPM, fast: 160 WPM)

- Edge cases like empty scripts are handled without errors

4.4.2 Integration Testing

Integration tests validate the interaction between multiple components and external services.

4.4.2.1 User Synchronization Flow Test

Test Case: Verify complete user authentication and Supabase synchronization flow.

```
// Integration Test: User Authentication and Sync

describe('User Authentication and Synchronization', () => {
  test('should authenticate user and sync to Supabase',
    async () => {
      // Step 1: Authenticate with Clerk
      const signUpResponse = await clerk.signUp({
        emailAddress: 'testuser@example.com',
        password: 'SecurePass123!',
        firstName: 'Test',
        lastName: 'User'
      });

      expect(signUpResponse.status).toBe('complete');
      const userId = signUpResponse.createdUserId;

      // Step 2: Get authentication token
      const session = await clerk.sessions.getSessionList()[0];
      const token = await session.getToken();

      // Step 3: Call sync endpoint
      const syncResponse = await fetch(
        'http://localhost:5000/api/users/sync', {
          method: 'POST',
          headers: {
            'Authorization': 'Bearer ${token}',
            'Content-Type': 'application/json'
          }
        }
      );
    }
  );
});
```

```
    },
    body: JSON.stringify({
      email: 'testuser@example.com',
      firstName: 'Test',
      lastName: 'User'
    })
  });

expect(syncResponse.status).toBe(200);
const syncData = await syncResponse.json();
expect(syncData.success).toBe(true);

// Step 4: Verify in Supabase
const { data: user } = await supabase
  .from('users')
  .select('*')
  .eq('clerk_id', userId)
  .single();

expect(user).not.toBeNull();
expect(user.email).toBe('testuser@example.com');
expect(user.first_name).toBe('Test');
expect(user.last_name).toBe('User');
});
});
```

4.4.2.2 Script Generation End-to-End Test

Test Case: Test complete script generation workflow from API request to database storage.

```
// Integration Test: Script Generation Workflow

describe('Script Generation Workflow', () => {
  let authToken: string;
  let userId: string;

  beforeAll(async () => {
    // Setup authenticated user
    const auth = await setupTestUser();
```

```
    authToken = auth.token;
    userId = auth.userId;
  });

test('should generate script and save to database',
  async () => {
    const scriptParams = {
      title: 'Test Educational Video',
      language: 'English',
      topic: 'Introduction to AI',
      scriptType: 'Educational',
      tone: 'Professional',
      targetAudience: 'Beginners',
      duration: 120,
      pacing: 'medium',
      includeHook: true,
      includeCTA: true
    };

    // Call generation endpoint
    const response = await fetch(
      'http://localhost:5000/api/content/generate-script', {
        method: 'POST',
        headers: {
          'Authorization': 'Bearer ${authToken}',
          'Content-Type': 'application/json'
        },
        body: JSON.stringify(scriptParams)
      });

    expect(response.status).toBe(200);
    const result = await response.json();

    // Verify response structure
    expect(result).toHaveProperty('scriptId');
    expect(result).toHaveProperty('content');
    expect(result).toHaveProperty('metadata');
    expect(result.metadata).toHaveProperty('wordCount');
    expect(result.metadata).toHaveProperty('estimatedDuration');
```

```
// Verify database entry
const { data: script } = await supabase
  .from('scripts')
  .select('*')
  .eq('script_id', result.scriptId)
  .single();

expect(script).not.toBeNull();
expect(script.user_id).toBe(userId);
expect(script.title).toBe('Test Educational Video');
expect(script.language).toBe('English');
expect(script.method).toBe('generated');
expect(script.status).toBe('draft');
});
});
```

4.4.3 Manual Testing

Manual testing was conducted to validate user interface functionality and user experience.

4.4.3.1 Dashboard Navigation Test

Test Scenario: User navigates through the dashboard and accesses different sections.

Steps:

1. Log in with valid credentials
2. Verify dashboard loads with user-specific data
3. Click “Create Content” in sidebar
4. Verify navigation to content creation page
5. Verify active route highlighting in sidebar
6. Test logout functionality

Expected Results:

- All navigation links function correctly
- Active routes are visually highlighted

- Protected routes redirect to login when not authenticated
- Session persists across page refreshes

4.4.3.2 Script Generation User Flow Test

Test Scenario: Complete script generation workflow from user perspective.

Steps:

1. Navigate to “Create Content”
2. Select language (English/Urdu)
3. Choose “Generate from scratch”
4. Fill in all required fields:
 - Title: “Social Media Marketing Guide”
 - Topic: “How to grow your Instagram following”
 - Script type: Educational
 - Tone: Conversational
 - Target audience: Small business owners
 - Duration: 2 minutes
 - Pacing: Medium
5. Enable “Include hook” and “Include CTA”
6. Click “Generate Script”
7. Wait for loading indicator
8. Verify generated script appears
9. Review word count and estimated duration
10. Test “Make it shorter” quick action
11. Verify script updates
12. Click “Save as Draft”
13. Verify success message

Expected Results:

- All form fields validate correctly
- Loading states appear during generation
- Generated script matches language and parameters
- Quick actions modify script appropriately
- Draft saves successfully to database
- User can retrieve draft later from dashboard

4.4.4 Security Testing

4.4.4.1 Authentication Security Test

Test Case: Verify that protected routes reject unauthorized access.

```
// Security Test: Route Protection
```

```
describe('Route Security', () => {
  test('should block access to protected API without token',
    async () => {
      const response = await fetch(
        'http://localhost:5000/api/content/generate-script', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ topic: 'test' })
        });

      expect(response.status).toBe(401);
    });

  test('should block access with expired token', async () => {
    const expiredToken = 'expired_jwt_token';

    const response = await fetch(
      'http://localhost:5000/api/users/sync', {
        method: 'POST',
        headers: {
          'Authorization': 'Bearer ${expiredToken}',
          'Content-Type': 'application/json'
        }
      });
  });
});
```

```

    }
  });

  expect(response.status).toBe(401);
});

test('should enforce user-specific data access (RLS)',
  async () => {
    // User A creates a script
    const userAToken = await getTokenForUser('userA');
    const scriptResponse = await createScript(userAToken,
                                              { title: 'A Script' });
    const scriptId = scriptResponse.scriptId;

    // User B attempts to access User A's script
    const userBToken = await getTokenForUser('userB');
    const accessResponse = await fetch(
      'http://localhost:5000/api/content/script/${scriptId}',
      {
        headers: { 'Authorization': 'Bearer ${userBToken}' }
      }
    );

    // Not found (RLS prevents access)
    expect(accessResponse.status).toBe(404);
  });
});

```

4.4.5 XTTS v2 Voice Cloning Testing

4.4.5.1 English Voice Cloning Test

Test Case: Validate zero-shot voice cloning with pre-trained XTTS v2 model for English.

Test File: TTS_Models/tests/test_english_xtts.py

```

import torch
from TTS.api import TTS
import pytest

```

```
class TestEnglishXTTS:
    @classmethod
    def setup_class(cls):
        cls.device = "cuda" if torch.cuda.is_available() else "cpu"
        cls.tts = TTS(
            "tts_models/multilingual/multi-dataset/xtts_v2"
        ).to(cls.device)

    def test_zero_shot_voice_cloning(self):
        """Test zero-shot voice cloning with reference audio"""
        test_text = "This is a test of voice cloning technology."
        speaker_wav = "test_data/reference_speaker.wav"
        output_path = "test_output/cloned_voice.wav"

        # Perform TTS
        self.tts.tts_to_file(
            text=test_text,
            speaker_wav=speaker_wav,
            language="en",
            file_path=output_path
        )

        # Verify output file exists and has content
        assert os.path.exists(output_path)
        audio_duration = get_audio_duration(output_path)
        assert audio_duration > 0
        assert audio_duration < 10 # Should be reasonable length

    def test_voice_similarity(self):
        """Test voice similarity between reference and
        synthesized audio"""
        reference_wav = "test_data/reference_speaker.wav"
        synthesized_wav = "test_output/cloned_voice.wav"

        # Extract speaker embeddings
        ref_embedding = extract_speaker_embedding(reference_wav)
        synth_embedding = extract_speaker_embedding(synthesized_wav)

        # Calculate cosine similarity
        similarity = cosine_similarity(ref_embedding, synth_embedding)
```

```

    # Voice similarity should be high (> 0.75)
    assert similarity > 0.75

def test_audio_quality(self):
    """Test synthesized audio quality metrics"""
    output_wav = "test_output/cloned_voice.wav"

    # Load audio
    audio, sr = librosa.load(output_wav, sr=22050)

    # Check signal-to-noise ratio
    snr = calculate_snr(audio)
    assert snr > 20 # Minimum acceptable SNR

    # Check for clipping
    max_amplitude = np.max(np.abs(audio))
    assert max_amplitude < 0.99 # No clipping

```

Expected Results:

- Synthesized audio file generated successfully
- Voice similarity score > 0.75 (high similarity to reference)
- Audio quality metrics within acceptable ranges (SNR > 20dB, no clipping)

4.4.5.2 Urdu Fine-tuned Model Test

Test Case: Validate custom fine-tuned XTTS v2 model for Urdu language with voice cloning.

Test File: TTS_Models/tests/test_urdu_xtts.py

```

import torch
from TTS.api import TTS
import pytest

```

```

class TestUrduXTTS:
    @classmethod
    def setup_class(cls):

```

```
cls.device = "cuda" if torch.cuda.is_available() else "cpu"
# Load custom fine-tuned model
cls.tts = TTS(
    model_path="custom_models/urdu/xtts_v2_finetuned",
    config_path="custom_models/urdu/config.json"
).to(cls.device)

def test_urdu_text_synthesis(self):
    """Test Urdu text-to-speech synthesis"""
    urdu_text = "          "
    speaker_wav = "test_data/urdu_reference_speaker.wav"
    output_path = "test_output/urdu_synthesized.wav"

    # Perform TTS
    self.tts.tts_to_file(
        text=urdu_text,
        speaker_wav=speaker_wav,
        language="ur",
        file_path=output_path
    )

    assert os.path.exists(output_path)
    audio_duration = get_audio_duration(output_path)
    assert audio_duration > 0

def test_urdu_tokenization(self):
    """Test proper tokenization of Urdu script"""
    urdu_text = "          "

    # Tokenize using extended vocabulary
    tokens = self.tts.tokenizer.encode(urdu_text)

    # Verify no unknown tokens
    unk_token_id = self.tts.tokenizer.token_to_id("<unk>")
    assert unk_token_id not in tokens

    # Verify tokens can be decoded back
    decoded_text = self.tts.tokenizer.decode(tokens)
    assert decoded_text == urdu_text
```

```

def test_urdu_voice_cloning_quality(self):
    """Test quality of Urdu voice cloning"""
    urdu_text = "          "
    speaker_wav = "test_data/urdu_reference_speaker.wav"
    output_path = "test_output/urdu_cloned.wav"

    self.tts.tts_to_file(
        text=urdu_text,
        speaker_wav=speaker_wav,
        language="ur",
        file_path=output_path
    )

    # Verify speaker similarity
    ref_embedding = extract_speaker_embedding(speaker_wav)
    synth_embedding = extract_speaker_embedding(output_path)
    similarity = cosine_similarity(ref_embedding, synth_embedding)

    # Urdu model should maintain voice similarity > 0.70
    assert similarity > 0.70

def test_dataset_preprocessing_validation(self):
    """Validate preprocessed Urdu dataset quality"""
    dataset_path = "TTS_Models/datasets/urdu_processed"

    # Load dataset metadata
    metadata = load_metadata(f"{dataset_path}/metadata.csv")

    # Check minimum samples
    assert len(metadata) > 1000

    # Verify audio files exist
    for row in metadata[:100]: # Sample check
        audio_path = row['audio_file']
        assert os.path.exists(audio_path)

    # Check audio duration
    duration = get_audio_duration(audio_path)
    assert 1 <= duration <= 20

```

```
def test_model_inference_speed(self):
    """Test inference speed for real-time requirements"""
    urdu_text = "      "
    speaker_wav = "test_data/urdu_reference_speaker.wav"

    import time
    start_time = time.time()

    wav = self.tts.tts(
        text=urdu_text,
        speaker_wav=speaker_wav,
        language="ur"
    )

    inference_time = time.time() - start_time
    # Assuming 22050 Hz sample rate
    audio_duration = len(wav) / 22050

    # Real-time factor should be < 1.0 for acceptable performance
    rtf = inference_time / audio_duration
    # Acceptable for non-real-time applications
    assert rtf < 2.0
```

Expected Results:

- Urdu text synthesized correctly without tokenization errors
- Voice cloning maintains similarity > 0.70 for Urdu speakers
- Dataset preprocessing validates minimum quality standards
- Inference time acceptable for production use (RTF < 2.0)

4.4.6 Test Coverage Summary

The testing strategy covers:

- **Unit Tests:** Individual functions for authentication, metadata calculation, prompt generation, tokenization
- **Integration Tests:** End-to-end workflows including user sync and script generation

- **Manual Tests:** UI/UX validation, user flows, cross-browser compatibility
- **Security Tests:** Authentication enforcement, authorization checks, data isolation
- **TTS Model Tests:** Voice cloning quality, Urdu language synthesis, dataset validation, inference performance

All tests are designed to ensure system reliability, security, and correct functionality across different usage scenarios. Automated tests can be run using:

```
# Run all tests  
npm test
```

```
# Run tests with coverage report  
npm run test:coverage
```

```
# Run specific test file  
npm test auth.test.ts
```

```
# Run XTTS model tests  
cd TTS_Models  
python -m pytest tests/ -v
```

```
# Run Urdu model specific tests  
python -m pytest tests/test_urdu_xtts.py -v
```


Chapter 5

Conclusions and Future Work

5.1 Conclusion

The development of the Urdu AI Video Creator has successfully established a robust, enterprise-grade platform for multilingual content generation. By integrating modern web technologies with advanced artificial intelligence models, the system addresses the critical gap in tools available for Urdu language content creators.

The implementation phase achieved several significant milestones. First, the system features a secure and scalable architecture built upon Next.js 14, Clerk authentication, and Supabase, ensuring data privacy and seamless user session management. Second, the AI-powered script generation workflow was successfully implemented using the OpenRouter API, allowing users to generate, refine, and iterate on scripts with context-aware metadata such as pacing and duration.

The most notable technical achievement of this project is the successful development of a custom text-to-speech pipeline for the Urdu language. Recognizing the limitations of existing commercial tools, the project implemented a fine-tuning pipeline for the Coqui XTTS v2 model. This involved aggregating multiple open-source datasets, including Common Voice and the NLP-Story2Audio dataset, and extending the model's vocabulary using Byte Pair Encoding (BPE) to properly handle Urdu script. Despite technical challenges posed by the depreciation of the Coqui library, a compatible training environment was established to produce a model capable of voice cloning with a similarity score exceeding 0.70.

Currently, the platform functions as a comprehensive audio-content creation tool. It effectively manages the workflow from user login and idea generation to script refinement and final multilingual speech synthesis. Testing has validated the system's security through route protection checks and confirmed the inference quality of both English (zero-shot) and Urdu (fine-tuned) voice models.

5.2 Future Work

While the current system successfully handles script and audio generation, further development is required to transform it into a complete video production suite. The future roadmap focuses on three key areas: User Experience (UX) enhancement, optimization of the Urdu voice cloning model, and the integration of visual synthesis.

5.2.1 Improving Urdu Voice Cloning

Although the current fine-tuned model achieves acceptable similarity scores, further optimization is planned to reach commercial-grade quality:

- **Dataset Expansion:** Incorporating a larger corpus of high-quality, single-speaker Urdu audio data to improve the model's prosody and emotional expressiveness.
- **Hyperparameter Tuning:** Conducting extensive experiments with training parameters to reduce the "robotic" artifacts occasionally present in long-form Urdu synthesis.
- **Inference Optimization:** Optimizing the model inference pipeline to reduce the Real-Time Factor (RTF) below the current threshold, ensuring faster generation times on standard hardware.

5.2.2 Lip Synchronization and Video Generation

The final phase of development will focus on bridging the gap between audio and video:

- **Visual Synthesis Integration:** Integrating Generative Adversarial Networks (GANs), such as Wav2Lip or similar state-of-the-art models, to synchronize the generated audio with static images or video avatars.
- **Video Rendering Pipeline:** Developing a backend service using tools like FFmpeg to merge the synchronized visual stream with the high-quality audio track.
- **Final Output Generation:** Enabling users to export the fully rendered video, completing the end-to-end pipeline from a text prompt to a lip-synced Urdu video.

By addressing these areas, the platform will evolve from a sophisticated audio-scripting tool into a complete, AI-powered video production studio tailored for the Urdu-speaking demographic.

Bibliography

- [1] Faiza Adeeba and Sarmad Hussain. Unlt: Urdu natural language toolkit. *Natural Language Engineering*, 28(1):1–38, 2022.
- [2] Kevin Banares. Ai-powered content creation: How artificial intelligence automates and streamlines the workflow. *UX Matters*, 2024.
- [3] Khaled Eltouney, Mahmoud Khalil, and Hazem Abbas. Voice cloning: Comprehensive survey. *arXiv preprint arXiv:2505.00579*, 2025. Reviews current voice cloning models’ static nature and limited personalization.
- [4] Khaled Eltouney, Mahmoud Khalil, and Hazem Abbas. Voice cloning: Comprehensive survey. *arXiv preprint arXiv:2505.00579*, 2025.
- [5] Alitu Creator Insights. The paradox of ai content creation in 2025. *Alitu Blog*, July 2025.
- [6] Deloitte Insights. 2025 digital media trends: Social platforms are becoming the new media. *Deloitte*, September 2025.
- [7] Md Tawkat Islam Khondaker, Abdul Waheed, El Moatez Billah Nagoudi, and Muhammad Abdul-Mageed. Evaluating large language models for urdu. *arXiv preprint arXiv:2407.04459*, 2024.
- [8] KR Prajwal, Rudrabha Mukhopadhyay, Vinay P Namboodiri, and CV Jawahar. A lip sync expert is all you need for speech to lip generation in the wild. In *Proceedings of the 28th ACM International Conference on Multimedia*, pages 484–492, 2020.
- [9] Zengyi Qin, Wenliang Zhao, Xumin Yu, and Xu Sun. Openvoice: Versatile instant voice cloning. In *Proceedings of the International Conference on Machine Learning*, 2023.
- [10] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.

- [11] Muhammad Ali Shafique, Kanwal Mehreen, Muhammad Arham, Maaz Amjad, Sabur Butt, and Hamza Farooq. Alif: Advancing urdu large language models via multilingual synthetic data distillation. *arXiv preprint arXiv:2510.09051*, 2025.
- [12] AgileSoftLabs Content Strategy. Creator ai os vs traditional content tools: The 2025 roi analysis. *AgileSoftLabs Blog*, January 2025. Reports creators use 8–14 platforms, wasting 3.2 hours daily on switching.
- [13] Databricks Team. Building a generative ai workflow for the creation of more personalized marketing content. *Databricks Blog*, 2024. Unified AI workflows for end-to-end content generation.
- [14] SparkCo Team. Ai in content creation: 2025 trends for developers. *SparkCo Blog*, September 2025.
- [15] Xintao Wang, Liangbin Xie, Chao Dong, and Ying Shan. Real-esrgan: Training real-world blind super-resolution with pure synthetic data. In *International Conference on Computer Vision Workshops (ICCVW)*, 2021.